



SEP2 Project Report



Loke Hansen
363299

Bátor Gábora
363085

Marko Stokic
361768

Mike Lorenzen
363259

Victor Tonu
363137

Supervised by:

Allan Henriksen

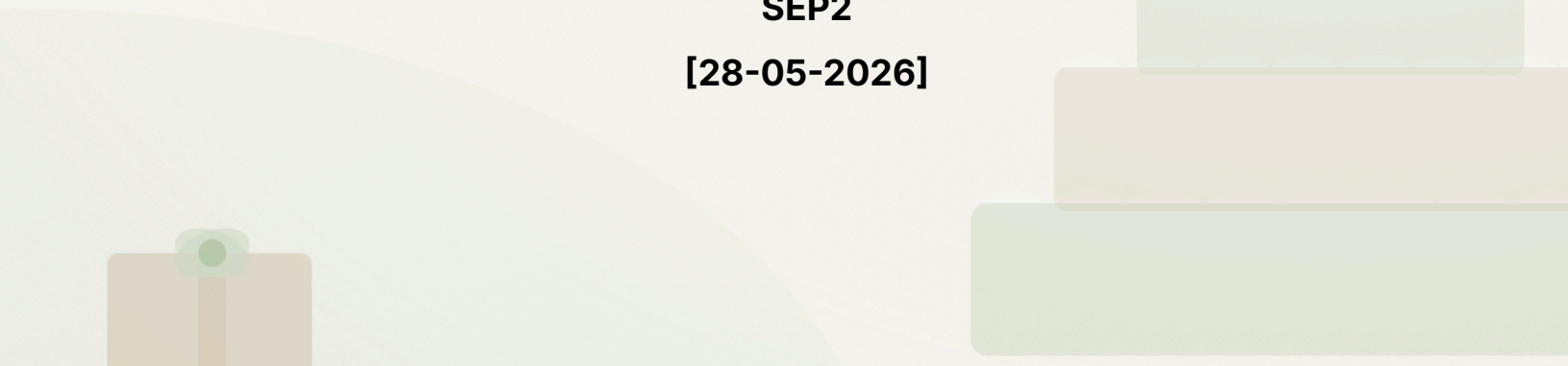
Joseph Okika

Number of characters: 66154

Software Technology Engineering

SEP2

[28-05-2026]



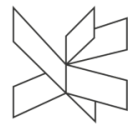
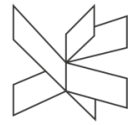


Table of contents

Table of contents	1
1. Abstract	4
2. Introduction	5
3 Analysis	6
3.1) Actor description.....	6
3.2) Functional & non-functional requirements.....	7
Functional requirements.....	7
Non-functional requirements.....	8
3.3) Use case diagram.....	9
3.4) Use cases.....	10
User:.....	10
Organizer:.....	10
Participant:.....	11
Use case description "Create party" full table.....	11
Relation between requirements and Use Case Diagram.....	12
3.5) Domain model.....	15
3.6) System sequence diagram.....	16
4. Design	18
4.1) Architectural diagram.....	18
4.2) Class Diagram.....	19
Client:.....	19
Server:.....	20
Shared:.....	20
4.3) MVVM Pattern Diagram.....	21
4.4) Sequence Diagrams.....	22
4.5) Design Patterns.....	24
4.5.1) TCP client-server.....	24
Server-side:.....	25
Client-side:.....	25
4.5.2) Observer.....	26
4.5.3) Singleton.....	27
4.5.4) SOLID Principles.....	29
S - Single responsibility principle:.....	29
O - Open/Closed principle:.....	29
L - Liskov substitution principle:.....	29



I - Interface segregation principle:.....	30
D - Dependency inversion principle:.....	30
4.5.5) Producer-Consumer pattern.....	31
4.5.6) OOP fundamentals.....	32
4.6) Graphical User Interface.....	33
4.7) DATABASE.....	37
4.7.1) EER Diagram:.....	37
4.7.2) GR Diagram:.....	38
5. Implementation.....	39
5.1) from GUI to Database and back.....	39
5.1.1) Overview.....	39
5.1.2) GUI.....	40
5.1.3) Controller.....	41
5.1.4) ViewModel.....	43
5.1.5) Model implementations.....	45
5.1.5.1) PartyModel (interface).....	45
5.1.5.3) PartyClientModel.....	46
Example method:.....	48
5.1.5.4) To conclude:.....	49
5.1.6) PartyClientHandler.....	50
5.1.7) Server – PartyClient.....	52
5.1.7.1) Server:.....	52
5.1.7.2) PartyClient:.....	53
5.1.8) Database Connection.....	55
5.1.9) PartyDAO.....	56
5.1.10) Party model class.....	58
5.1.11) Threading across the application.....	58
6. Testing.....	60
6.1) Unit testing.....	60
6.2) Integration testing.....	66
6.3) System test.....	69
6.4) Testing summary.....	70
7. Results.....	72
8. Discussion.....	76
9. Conclusion and Recommendations.....	78
11. References.....	80



1. Abstract

The project seeks to investigate "How can an organizer and participants work together to share the burden of event creation among themselves and streamline the management of an event?"

To address this problem the group 4 team has developed the Party Planner application. The system is a multi-user system that implements party hosting, date voting, item contribution and a chat system.

The system was implemented in Java and was constructed based on the MVVM design pattern, while the database was written in SQL, and the client-server structure was constructed based on patterns such as Singleton, Observer, Proxy and DAO. The system was constructed on 17 functional requirements and 4 non-functional requirements. 15 out of the 17 functional requirements were accomplished while all of the non-functional requirements were met. The testing section demonstrates that the main functionalities of the system are functioning as intended, although some additional testing would be required to further verify the systems functionality. Overall, the project indicates that a centralized planning approach will reduce the burden of event organization.



2.Introduction

The planning of a party is dependent on a couple of factors, such as communication with the participants, endless spreadsheets, whitelists, time planning, and inventory management. Some applications support these struggles, but they lack a unified structure and the ability to assign responsibilities. This burden is often placed on just one person, making party planning a headache, and demotivating. Recent studies on social events show that coordinating large scale events can become overwhelming and inefficient without a structured layout, this can discourage the organizer and lead to plans dissolving (Tervooren, T. 2019, January).

Current party planning applications like "CHECKIN" lack quality of life features, like participants having the option to vote for the date that the party occurs. Along with the option to bring necessary items to the party. This project intends to add these quality-of-life features.

The project is based on this problem statement: "How can an organizer and participants work together to share the burden of event creation among themselves and streamline the management of an event?" The project looks further into these questions by deploying a multi-user system that is set to manage these shortcomings of modern applications. The software is called Party Planner and is designed to minimize the toll that event planning takes on the organizer. This is done by making major planning parts such as date picking and item coordination, a shared goal for every participant. This way the responsibility is not just on one person, but the whole party. From a practical viewpoint this makes the project relevant in the real world.

The project is academically relevant since it involves several software engineering principles such as use case modelling, database integration and a multi-user system. The project involves structured development methods through unified process and scrum.



3 Analysis

In the analysis portion of this report the requirements of the system are explored and elaborated on, its actors, and its general use structure. The actors and their roles within the system are briefly described followed by all the requirements of the system in order of importance. The use cases are also listed with a brief explanation and a table showing the relationship between requirements and use cases, a more detailed example of one of the use cases is also provided along with a use case diagram and a domain model.

3.1) Actor description

A brief explanation of the type of actors which interact with the system and what features they have access to.

User: The user is an actor which interacts with the exploratory functions of the system. They can browse parties and their details in addition to receiving invitations to parties, the user can join any party they see and become a participant, alternatively they can create their own party and become an organizer. A user can also add other Users as friends.

Participant: A user who joins a party that they didn't create becomes a participant. Participants can interact with parties in a more comprehensive manner. They are able to select items they wish to bring to the party as well as voting for the time of the party from the options available. A participant can also send text messages to other participants within a party.

Organizer: A user who creates a party becomes an organizer, they gain the ability to modify everything about their own party, they can rename it change the party's location or description, add items they wish the participant to bring, as well as adding options for when their party can be held for the participants to join, they can also invite participants or choose to make the party public. They can also delete the party, in addition to all of these capabilities an organizer can do everything a participant can.



3.2) Functional & non-functional requirements

The requirements of this system were carefully considered, with the goal of simplifying and reducing the friction of organizing parties, in addition to researching comparable publicly available software solutions.

Functional requirements

1. As a user, I want to create a party, so I can structure the event for several participants.
2. As a user, I want to be able to create a username and password, so that I can have a presence on the app.
3. As a user, I want to be able to log in to my account, so I can have a personalized experience.
4. As an organizer, I want to add participants to the event, so they can see the event.
5. As a participant, I want to see the parties I'm invited to and join them, so I can interact with the party organization features.
6. As an organizer, I want to be able to edit the party, to manage the item list and the description of the event.
7. As an organizer, I want to be able to set a list of needed items to a party I've created, so that the participants can help me bring the items needed.
8. As a participant, I want to be able to mark off the items I would bring, so that the organizer and other participants don't bring duplicates.
9. As an organizer, I want to be able to choose dates for the participants to vote for, so that everyone has a say on the date of the party.
10. As a participant, I want to be able to vote for my preferred date of a party, so I can have a say in the planning.



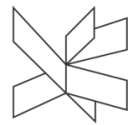
11. As a participant, I want to be able to leave parties I have already joined, as if something comes up this way my intentions are clear.
12. As an organizer, I want to remove participants from the event, so I can manage the amount of participants.
13. As a participant, I want to be able to use the party chat, so I can communicate with the rest of the party.
14. As a user, I want to be able to add other users as friends, so that I can more easily interact with them through the system.
15. As a user, I want to be able to remove a friend.
16. As a participant and organizer, I want to receive a random small quest at the start of the party, so the party will have entertainment.
17. As a User I want to be able to change my username

Non-functional requirements

A list of the systems non--functional requirements ordered by their importance, considered by their importance to the functionality of the system their technical feasibility and the overall maintainability of the system,

[1-Most important 21-Least Important]

18. The system shall implement a multi-user system.
19. The system shall follow the MVVM work pattern.
20. The database shall be made with SQL.
21. The system shall be implemented in Java.



3.3) Use case diagram

The use case Diagram below visualizes the interaction between the actors of the system and the use cases as well as the interaction between actors.

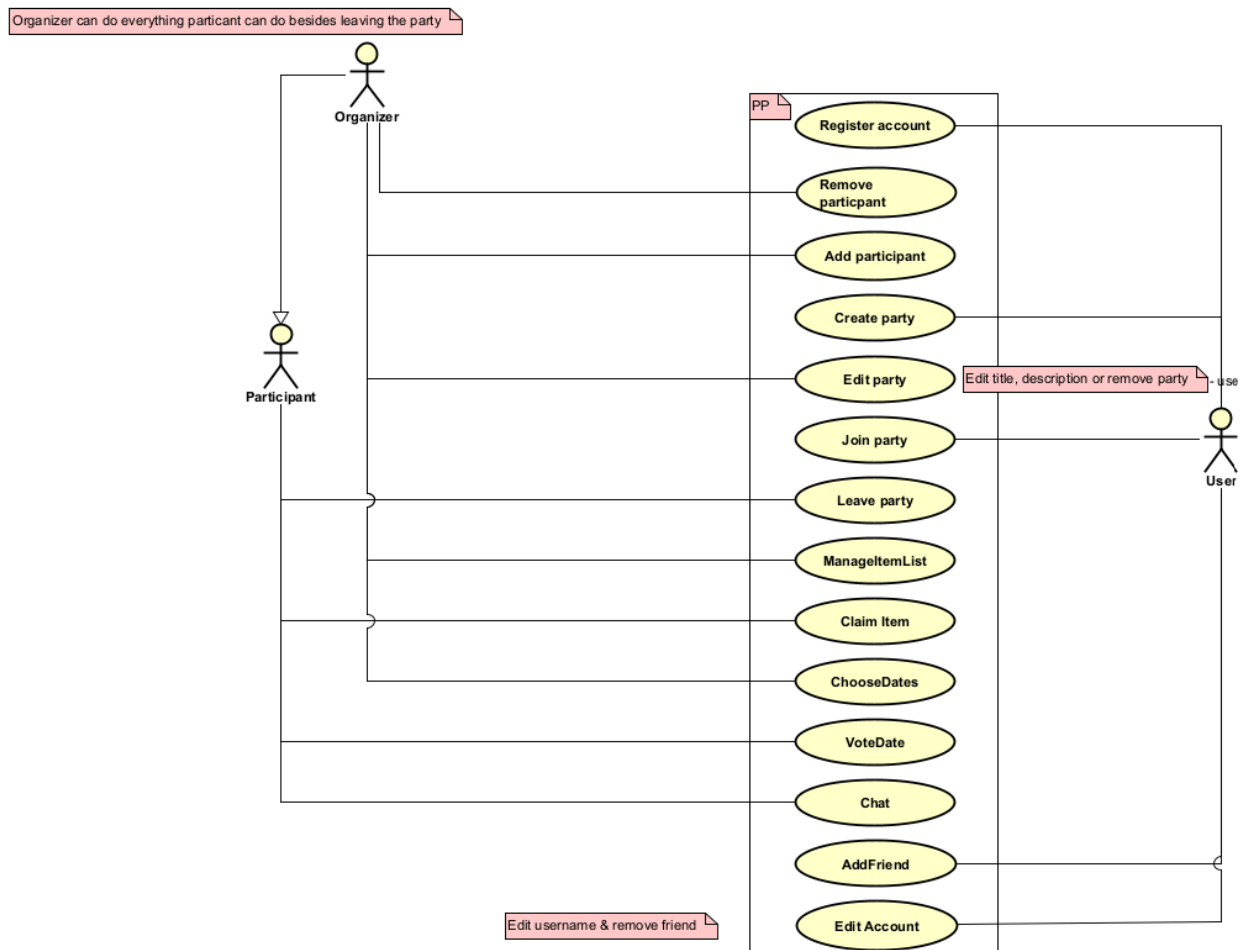


Figure 3.1 Use Case Diagram



3.4) Use cases

User:

Create Account: This use case allows a non-descript user to declare their email username and password so they can have access to the rest of the system

Join party: This use case allows a user to join a party that they're invited to which then turns them into a participant.

Add friend: This use case allows a User to add another User as a friend.

Organizer:

Remove participant: This use case allows an organizer to remove a participant from the party they manage

Add participant: The ability for the organizer to add any User as a participant to their party

Create party: This use case grants a User the ability to create a party with a name description location and optionally a date. After creation the user becomes an organizer within the party(refer to **figure 2.2** for an expanded view of this use case)

Manage party: This use case allows the organizer to change any details about their party and add items or date options.

Manage item list: This use case allows the organizer to remove or add wanted items to the item list.



Participant:

Leave party: This use case allows a participant to leave a party.

Mark item: This use case allows a participant to mark an item they wish to bring which is then displayed to all other participants.

Vote date: This use case allows a Participant to vote for a time they want. The live results of the voting are displayed to all users.

Chat: This use case allows a participant to send text messages to all other participants.



Use case description "Create party" full table

An example of a use case table which elaborates on how the create party use case functions in detail and aids in understanding the use case.

Note: For a full-format version of all the use cases refer to attached appendix D

Use case	Create party
Summary	Create a party with the necessary data
Actor	User
Precondition	The user is logged in and the system is capable of recording new parties
Postcondition	A party is added with the user creating it as the organizer
Base sequence	1. The system shows a list of current parties with party data (highlighting whether the user is an organizer or participant in each specific one) 2. The user selects "create party" option 3. The user fills in the necessary data for the party (name, description, location, and date.) 4. User presses create party 5. The user is now the organizer of the party 6. System creates party
Alternate sequence	[ALT0] The user can cancel the use case at any time – no changes are made and use case ends
Note	

Table 3.1 Expanded use case table for the "Create Party" use case

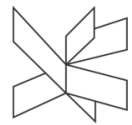


Relation between requirements and Use Case Diagram

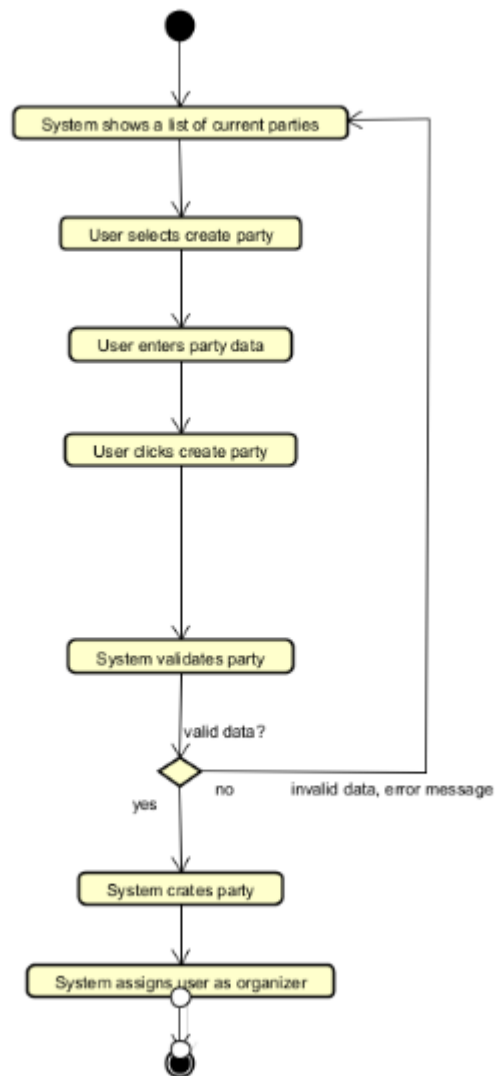
A table that shows how the requirements are related to the use cases present in the system, the requirements are represented by numbers which you can reference from the functional requirements

Use case	Covered requirements
Create party	3
Create Account	1, 2
Remove participant	12
Add participant	4
Manage party	6
Join party	5
Leave party	11
Manage item list	6, 7
Mark item	8
Choose Date	9
Vote date	6, 10
Chat	13
Add friend	14
Edit Account	15, 17

Table 3.2 relation table between the functional requirements and use cases



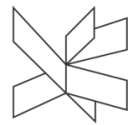
"Create Party" use case diagram



e

Figure 3.2 *"Create Party" use case diagram*

An astah diagram which illustrates the flow of the "Create Party" use case and its alternatives, this diagram portrays the interaction of a user with the system.



3.5) Domain model

The domain model portrayed below offers a comprehensive understanding of the entities and actions in this system and the interactions between them. The entity at the core of the diagram is the **Organizer** due to their ability to modify most of the systems utility, as seen bellow it can create an **event**, choose a **date** and **items** and invite **participants**, the other entities are visualized in the same manner bellow

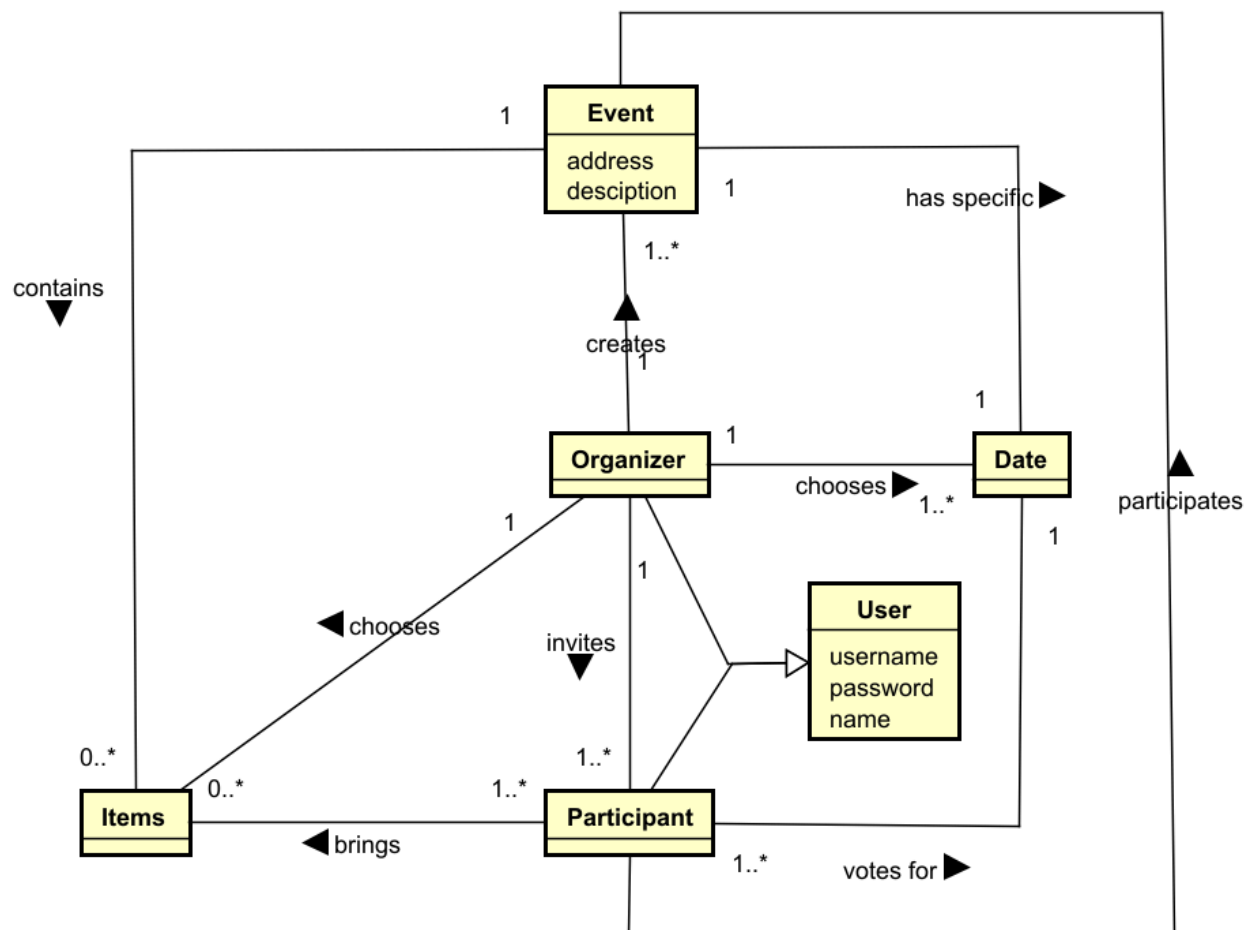
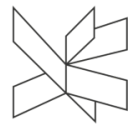


Figure 3.3 Domain model diagram



3.6) System sequence diagram

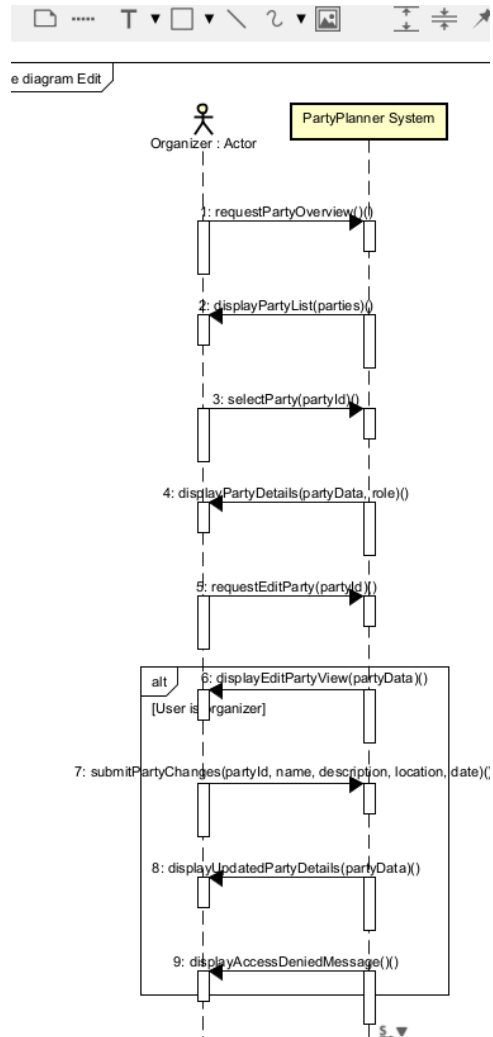
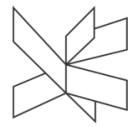


Figure 3.4 System sequence diagram for "Edit"

The user story in this diagram (Figure 3.4) is: As an organizer, I want to edit my party details so that I can update the event information for participants. The user chooses the party from his party list. If the user is the organizer the system allows him to edit the party, if the user is a participant the system doesn't allow the user to edit.



4. Design

The design section of this report is based on the foundation of how the Party Planner app was designed based on design choices such as the MVVM pattern, client-server communication, and other relevant design patterns that ensure separation of responsibilities and maintainability in the system. It also includes the design of the database and the design choices shown in the graphical user interface.

4.1) Architectural diagram

The main purpose of the architecture for the system is to showcase the TCP connection that combines the server side with the client side, this is based on a socket connection as shown here in the diagram below.

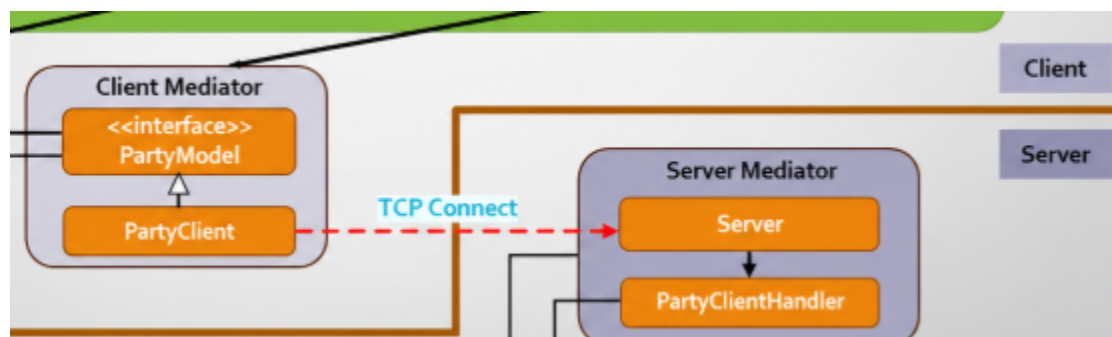


Figure 4.0 System architecture Diagram

The diagram is a UML class diagram for a system architecture, organized into several packages and layers. The top layer is the 'user' package, which contains several interfaces and concrete classes. The 'controller' package contains concrete classes that implement the interfaces. The 'view' package contains concrete classes that implement the interfaces. The 'database' package contains concrete classes that implement the interfaces. The 'service' package contains concrete classes that implement the interfaces. The 'entity' package contains concrete classes that implement the interfaces. The 'model' package contains concrete classes that implement the interfaces. The 'presentation' package contains concrete classes that implement the interfaces. The 'business' package contains concrete classes that implement the interfaces. The 'infrastructure' package contains concrete classes that implement the interfaces. The 'application' package contains concrete classes that implement the interfaces. The 'system' package contains concrete classes that implement the interfaces. The diagram shows a complex set of dependencies between these classes, with many classes depending on multiple interfaces and other classes. The diagram is a detailed representation of the system's architecture, showing the relationships between the different components and how they interact with each other.

This illustration of the application is clearly showing the architectural structure that was created for the program, it explains and shows how the diagram is divided into 3 different main packages, with those having sub packages. The three main packages mentioned are:

The client package has 3 different sub packages named view, viewModel and mediator. For the view package it includes all of the view controllers that the program is using for the different viewModels that they are connected to in the sub package named



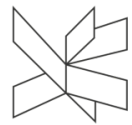
viewModel, the `fxml` files included in this package are also connected to each of their own controllers as well. This MVVM structure is further explained in the next section. The last sub package inside of the client side is the mediator, this package is used to contain the classes which handle the TCP connection and communication between the sockets. These classes are known as `PartyClientModel` and `PartyClient`.

Server:

For the server-side package the same ideology is used, where 3 sub packages include mediator, dao and database. For the mediator package, it handles the multi-threading part of the program inside of the `server` class, since the class listens for client connections and then creates a new thread for every connection by using the other class represented inside of the package, the `PartyClientHandler`. There is also a class called `ModelManager` inside of the server package which is used to implement the data from the `partyModel` interface that is shown in the last package shared. For the database package it only contains one class, which is the `dataBaseConnection` which acts like a singleton, connected to every single dao class which contains database queries.

Shared:

For this package the idea behind it is to include all of the classes that are shared or used by both the client and the server side. This means that it includes the main interface `partyModel` and the rest of the model classes.



4.3) MVVM Pattern Diagram

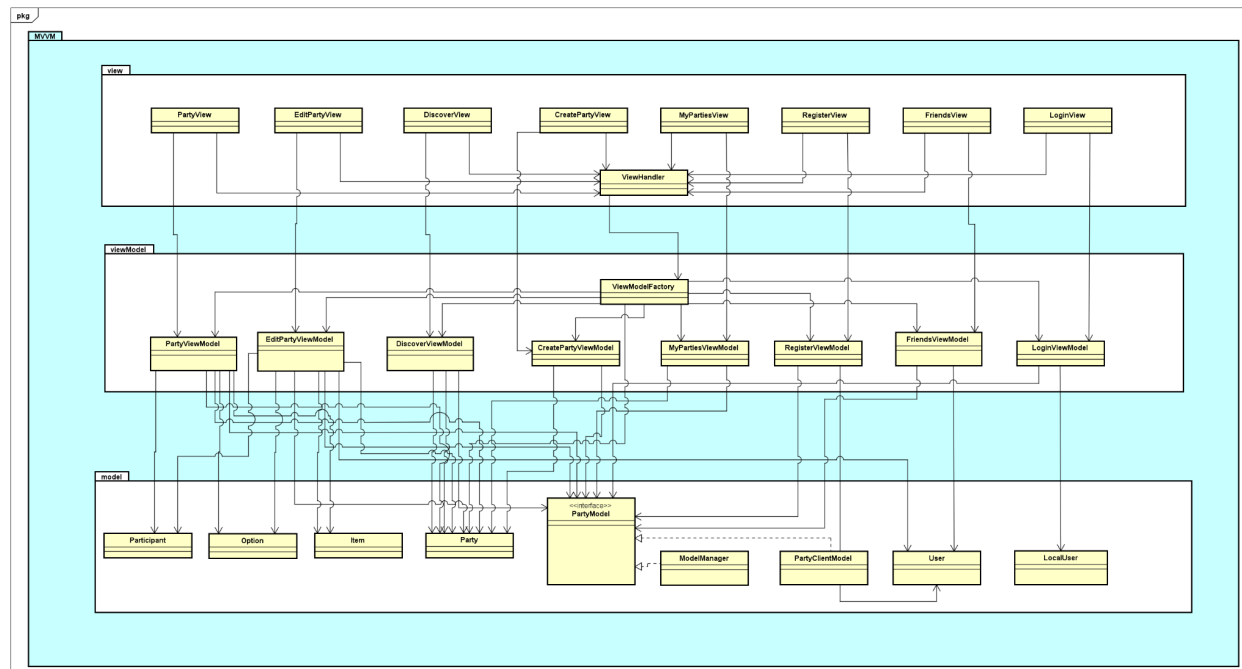
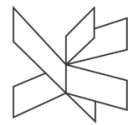


Figure 4.2 MVVM pattern diagram

The MVVM pattern diagram (Figure 3.3) shows the core MVVM (Gossman, 2005) structure for the project. The view layer contains the different views and the ViewHandler, the views are connected to their own separate View Models and the ViewHandler. The ViewHandler is connected to the ViewModelFactory which is responsible for creating the correct ViewModels, which connect the views to the business-logic in the model layer.



4.4) Sequence Diagrams

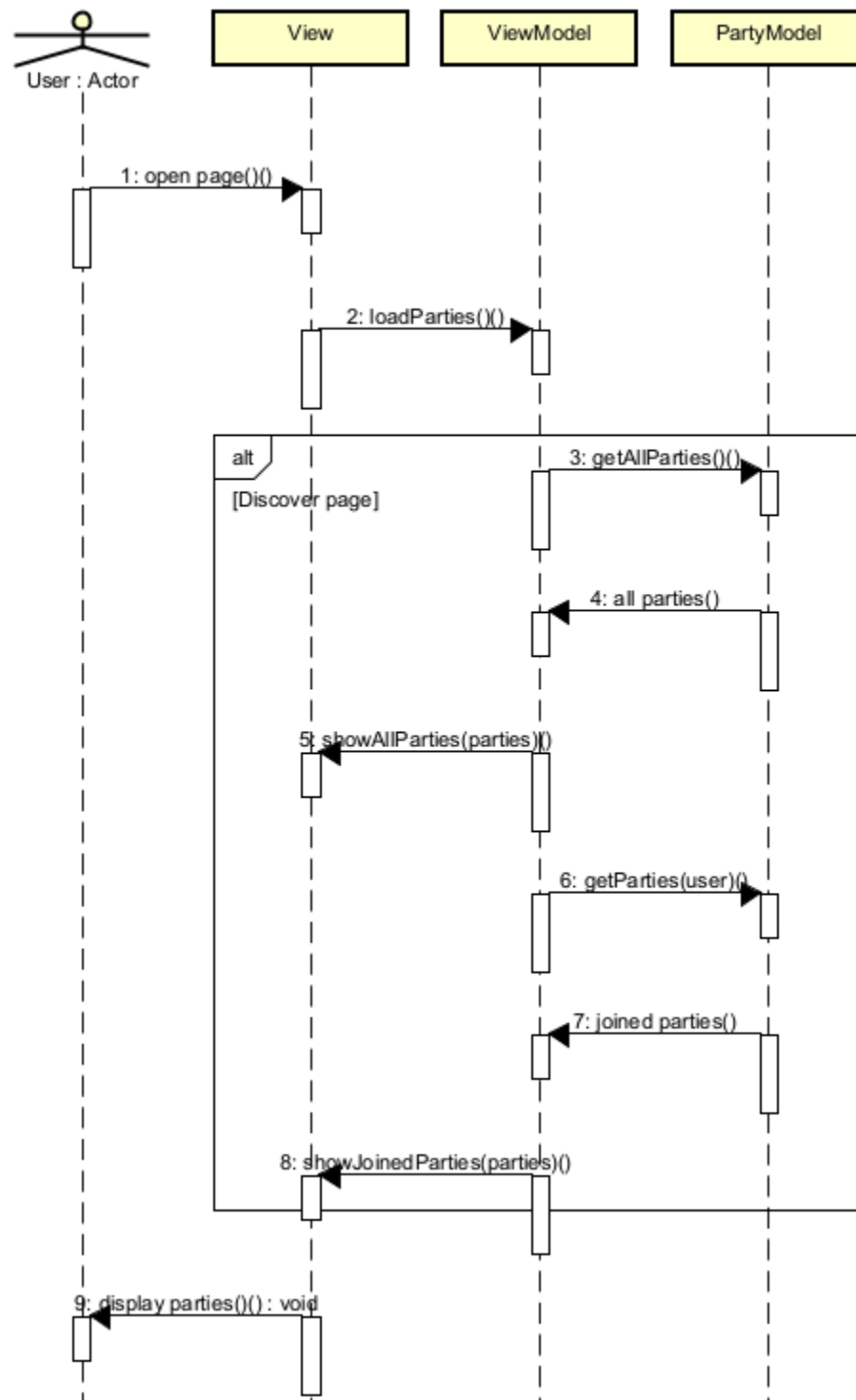
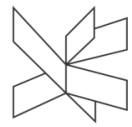


Figure 4.3 "getAllParties" sequence diagram



The sequence diagram (Figure 3.2) shows how the data of the parties are viewed in the different views. When the user opens the `discoverView` page, the view calls `loadParties()` in the viewmodel, the viewmodel then gets the data from the `PartyModel` interface. The flow depends on which page is being opened, if the `discover` page is being opened, the data needed are all parties in the system, but if the user is on the `My parties` page, the data show should be the current users' parties.

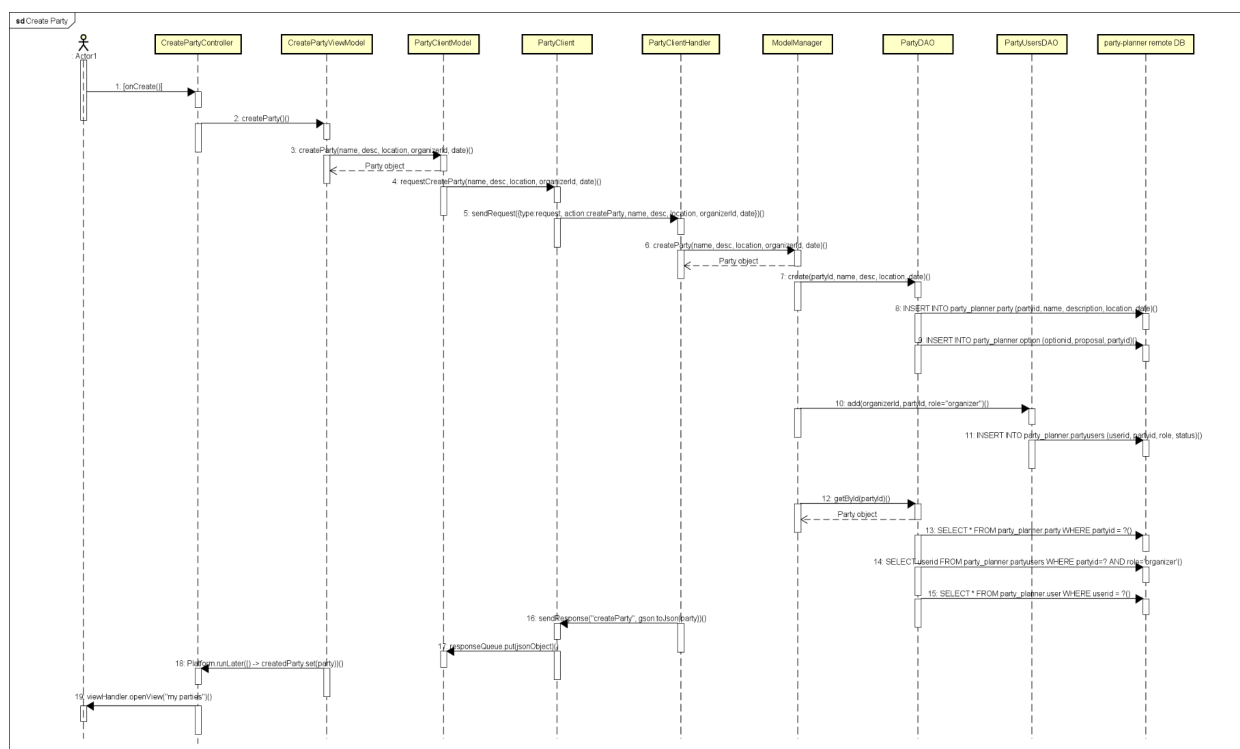


Figure 4.4 "CreateParty" sequence diagram

This sequence Diagram follows the typical path taken throughout the system, in this case the `CreateParty` use case is followed all the way from a button press by the user to the database and back. Allowing a reader to comprehend the flow of methods through the classes which can at times be overwhelming to understand through the class diagram alone.



4.5) Design Patterns

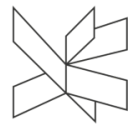
For this section the topic will be design patterns, and how these patterns have been implemented in the PP project (Party Planner). Each pattern will be explained in detail containing examples using diagrams to illustrate the implementation.

4.5.1) TCP client-server

The reasoning behind using the TCP client-server foundation is for the benefit of making the application able to communicate between multiple clients in real time while also having a way of delivering reliable data between these clients.

The choice between TCP and UDP is clear for this application since as explained, TCP will deliver data reliably and persistently throughout the connection compared to UDP which is less consistent (Kurose & Ross, 2021) . This also makes the server able to update the whole application in real time without the client having to permanently ask for updates to the server.

The client-server is set up through "Mediator" classes and packages as shown and explained here:



Server-side:

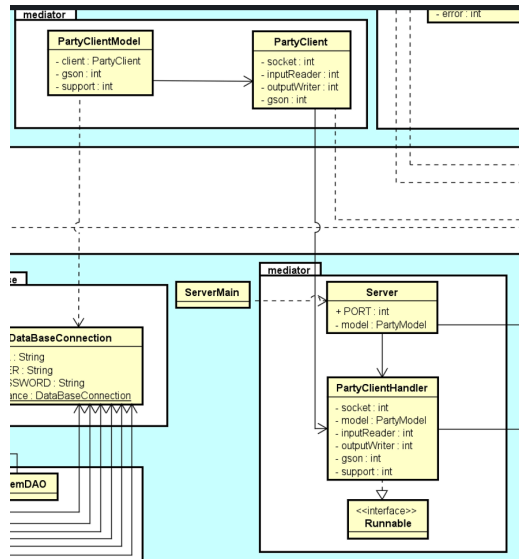


Figure 4.5 server-side mediator

Client-side:

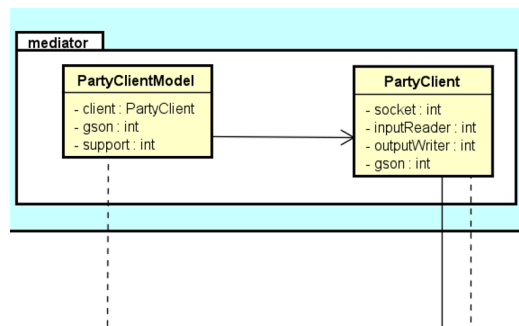
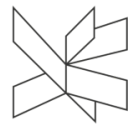


Figure 4.6 Client-Side Mediator

This package again has two different classes, the `PartyClientModel` and `PartyClient`. The `PartyClient` is making a new client-socket that then will create a connection from the client to server, the class also is responsible for sending requests and receiving responses through GSON. The class will send its requests through a `PrintWriter` named as `outputWriter`, while receiving its responses through a `BufferedReader` named as `inputReader`. The job of the `PartyClientModel` class is to act like some sort of bridge between the logic of the `PartyModel` interface in which it implements and connects the client-side and connects the view models.



4.5.2) Observer

For the Party Planner system the observer pattern that is implemented is crucial for the whole flow of the system. The system is a multi-user system, so when a change happens from one user, the system needs to update the UI to show the update for the rest of the users. The observer pattern makes sure that the system is able to show these changes without having to manually update in the code every time a change is done, thus meaning that without the observer pattern the system would have to manually refresh after every change like adding a participant or marking an item on the list of items.

The system includes a total of 8 view models which all receive "notifications" from the `PropertyChangeListener` which it implements and then overrides the `PropertyChange` methods in each of the view models, while the `ModelManager` is the broadcaster in this case, sending this notification to the rest of the view models, so they can automatically refresh the UI and update.

`ModelManager` does this by firing a `PropertyChangeEvent` while using the `PropertyChangeSupport` class from Java.

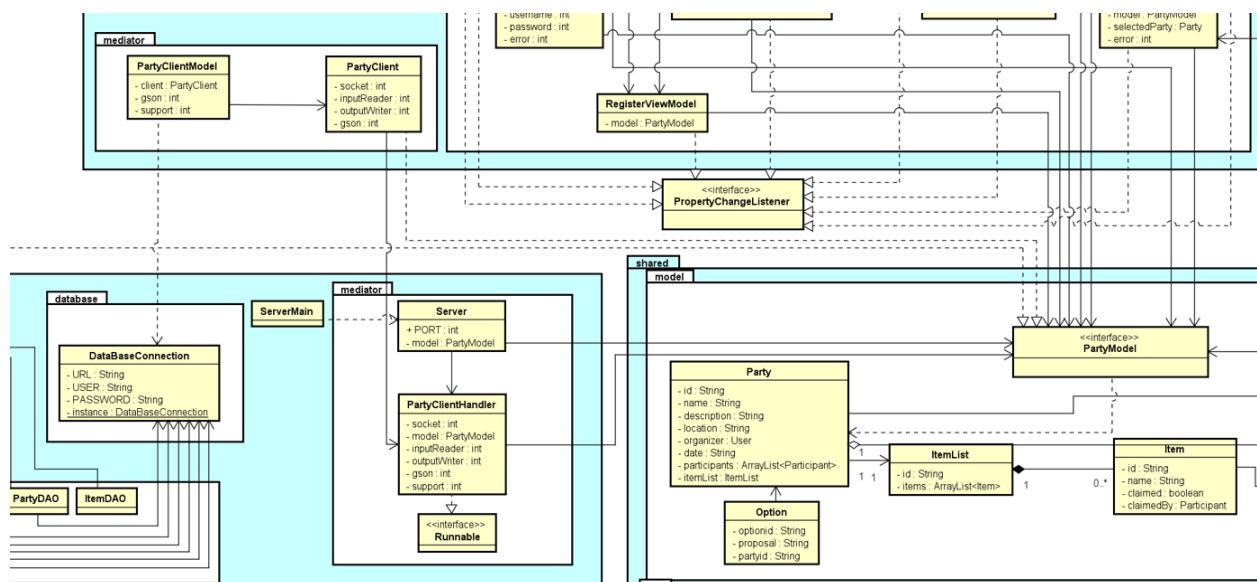


Figure 4.7 Observer Pattern illustrated



This figure illustrates the connections and how the observer pattern flows:

ModelManager: the broadcaster, uses `PropertyChangeSupport` to fire `PropertyChangeEvent` + implements the `PartyModel`.

PropertyChangeListener: the observer interface, used to connect all of the viewmodels, by implementation.

PartyModel: `partymodel` interface acts as the main interface extending every interface and implementing the `modelmanager`.

ViewModels: 8 total which all observe and implement `PropertyChangeListener`, so they can use `PropertyChange()`.

4.5.3) Singleton

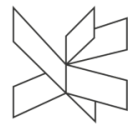
In the party planner application, the class `DataBaseConnection` is using the implementation of a singleton pattern in a way of the fact that the class is built with a:

- `Private static getInstance()` to make sure that the application is only using one instance of the time, the method is also synchronized to make sure that only one thread of a time can access the critical section.
- A private constructor.
- A lazy method called `getConnection()`, to make sure the connection will reopen automatically since the cloud database will close automatically after a while.



The sole reason behind using this pattern is to have a global access point, and to make sure that the application does not get overwhelmed with data from the DAO, since the application now only creates one instance instead of an instance for every method called.

27



4.5.4) SOLID Principles

The solid principles are a set of design principles that prioritise organized software that is easy to understand, maintain and extend in a work environment where many programmers work on the same code. The project is structured around these **SOLID** principles by separating logic from the interface and generally implementing patterns that reduce tight coupling.

S - Single responsibility principle:

The system is designed to separate responsibilities into many classes and not keep interface logic, business logic and application logic in the same classes. This principle is applied in `LoginController` where only interface logic applies, `LoginViewModel` where only application logic is applied, and `PartyDao` where the database access is.

O - Open/Closed principle:

The system is set out to utilise the **open closed** principle by making the system open for extension but closed for modification. This is done by implementing an interface "PartyModel" that both the `PartyClientModel` from the client side and `ModelManager` from the server-side implements, this allows functionality to be provided in different ways. Making it easy to extend without requiring major changes to the classes.

L - Liskov substitution principle:

The system utilises the Liskov Substitution principle with the `PartyModel` interface. Classes depending on `PartyModel` are able to work with any implementation of the interface, without needing to know whether it is using the `ModelManager` or the `PartyClientModel`. This way one implementation can be substituted for another while keeping the systems behaviour in check.



I - Interface segregation principle:

This principle lists that the client should not be forced to depend on methods it doesn't use. That is implemented through several interfaces that manage narrow sections of the functionality ensuring that the client does not implement extra unused methods.

D - Dependency inversion principle:

"High-level modules should not depend on low-level modules. Both should depend on abstractions" (Janssen 2024) The system follows the principle since high level modules like the ViewModels depend on the `PartyModel` interface, instead on the low-level modules `PartyClientModel` and `ModelManager`. But the low-level modules do not fully depend on the `PartyModel` interface, they directly use certain DAO's and concrete database connection code.

The system follows multiple of the **SOLID** principles partially, but there is room for improvement in this aspect.



4.5.5) Producer-Consumer pattern

The core fundamental idea for using the **Producer-Consumer pattern** in the party planner application is so the application will be able to handle all server responsibilities such as responses in a thread-safe way (Goetz et al., 2006) . It is done by decoupling the production part of the data from the rest of data, this meaning the consumption part of the data. It is done by using a buffer that the two different data types share in the middle.

For this application the buffer is clear, Party Planner is using a **LinkedBlockingQueue** field inside of the `PartyClientModel` class. The reasoning behind this choice is because this field does not need synchronization, it will automatically wait if the queue is empty.

For the class there is also a producer, in this case the listener thread that is built inside of the constructor of the class. This is the core that will listen for messages that server-side class sent and will then put this message in the back of the queue.

On the other hand there is also a consumer present in the case of the method `sendAndReceive()` which does one simple thing, its job is simply to take the response that is presented in the queue, if there is not any available it will wait.

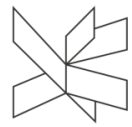
The pattern is only used for the simple fact that if it is not used there would be a guarantee that multiple threads would try to access the socket at the same time, reading the different responses of each other.



4.5.6) OOP fundamentals

Described by Oracle "**Object-oriented programming** is a method of programming based on a hierarchy of classes, and well-defined and cooperating objects"(Lesson 8: Object-Oriented Programming, n.d.). The system's foundation is based on **encapsulation, abstraction, inheritance** and **polymorphism**.

Encapsulation is used by implementing private fields, public setters and getters in the model layer. Abstraction is used by adding the `PartyModel` interface, while classes like `PartyClientModel` and `ModelManager` implements this interface thereby integrating polymorphism. Inheritance was not applied since the system mainly relies on implementation of interfaces.



4.6) Graphical User Interface

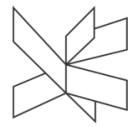
The party planner application is based on 8 different interface sections, those all being applied through the common use of the **JavaFX** functionality, This makes the user interface apart from the logic and follow the **MVVM** structure where the main controls used are the UI controls from this feature such as, `Button`, `Labels`, `DatePicker` and `ListView`.

In the following section, the use case Create Party will be described in detail, and how it is structured in the application through controls.

Create Party use case



Figure 4.9 "My parties" page GUI



The first part of the use case is showing the list of current parties for the user with an option in the top right corner to "+ create party".

Once this `CreateButton` controller button is pressed a new view with the `CreateParty` options will appear.

Figure 4.10 "Create Party" page GUI

This view has different controls that needs to be interacted with to fulfil the use case, these controls are described as:

- `TextField nameField`: this control is the text field that needs to be filled in below the "name" section.



- `TextField descriptionField`: this control is the text field below the "description" section that is not necessary to be filled but is an option that can later be filled in as well.
- `TextField locationField`: this is the control right below the latter that is designed for the purpose of filling in the information about the location, this is also mandatory to fill in.
- `DatePicker datepicker`: is a datepicker control that is meant to set the initial date of the party, this class lets you choose a date from the calendar, which has to be in the future.
- `Button createbutton`: lets the user create the party with the infilled information regarding the party.
- `Button backButton`: lets the user return to the previous view seeing the parties the user is already a part of. Is shown as "back to my parties".

After the `Button createbutton` is pressed with information that is all valid and following the rules regarding creating a party, a party is then created with the user creating it returned to the previous view, "my parties" view where the new party should also be displayed.



Loading screen

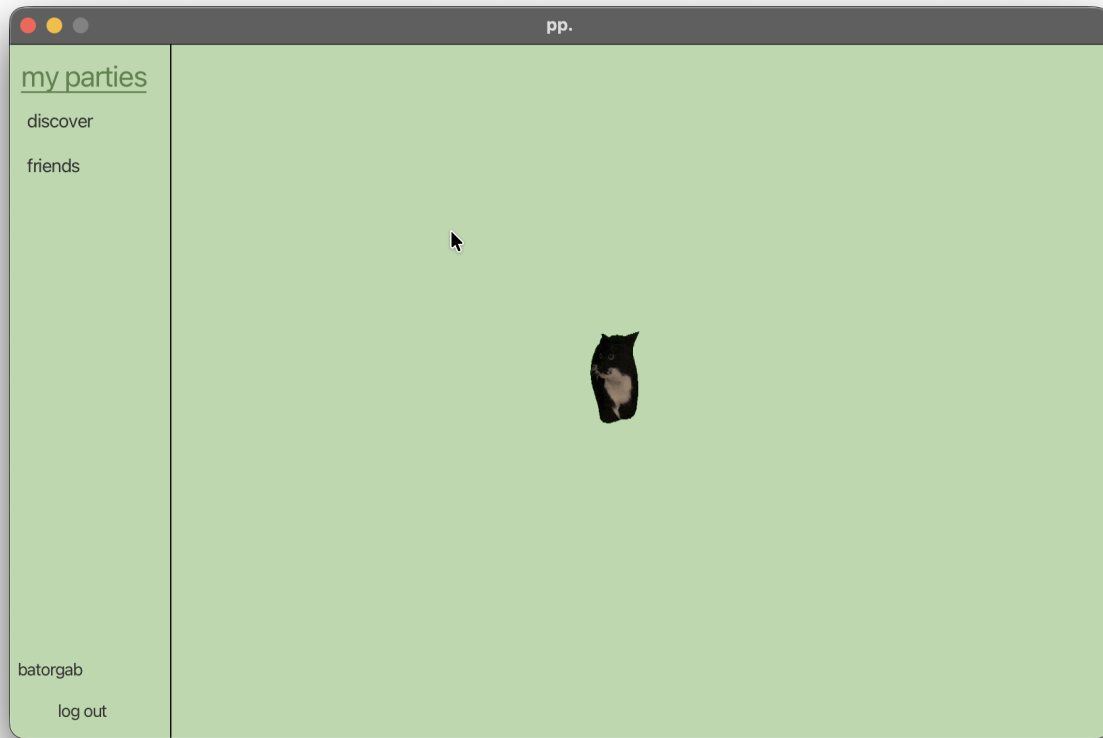
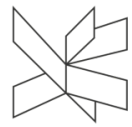


Figure 4.11 "my parties" page GUI with loading

All through the views, whenever the first network request is being sent and the program is waiting for the data coming in the response, a spinning cat animation (gif) is displayed with all other **FXML** elements hidden until the page loads. (More on this can be found in the Controller segment of the Implementations section).



4.7) DATABASE

The following section shows and briefly explain EER (Enchanted Entity Relationship) and GR (Global Relationship) diagrams for the database.

4.7.1) EER Diagram:

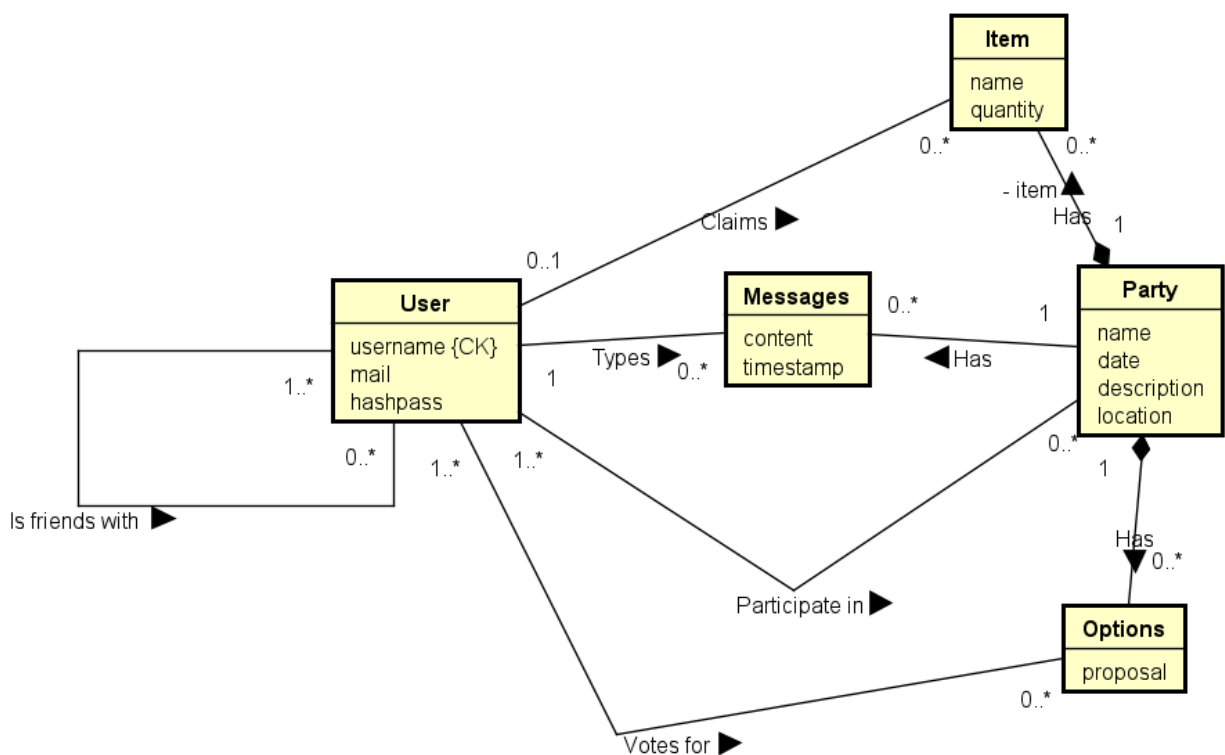
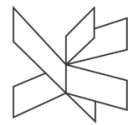


Figure 4.12 ERR Diagram of database

The database was designed based on the model (Figure 3.8). The diagram is based on the core entities User, Options, Items, Party and Messages.



4.7.2) GR Diagram:

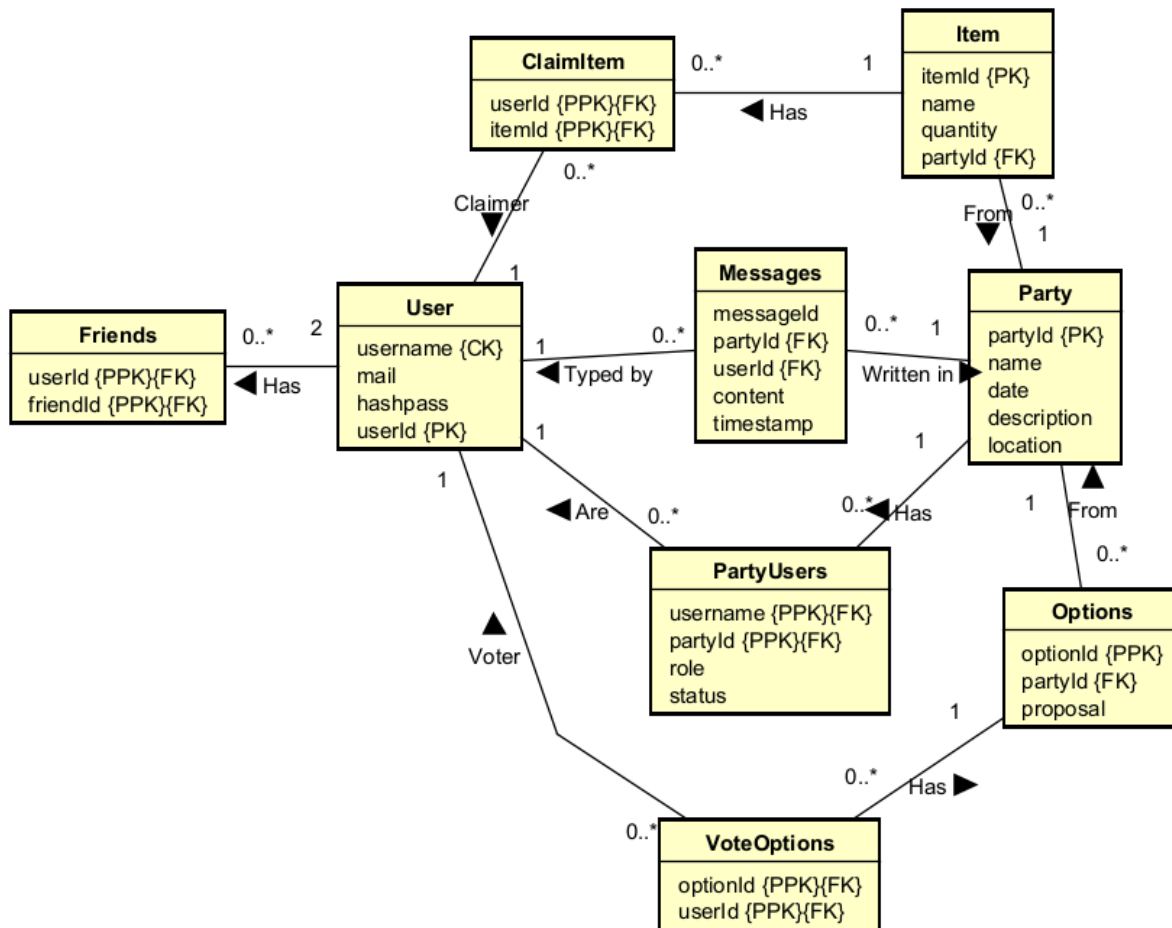


Figure 4.13 GR Diagram of database

Here is the **GR** diagram of the database, it is similar to the **EER** diagram but this version shows the different key constraints, like partyId as the **primary key**, username as a **candidate key** since username has to be unique, and foreign keys like partyId {FK} inside of item since items are specific to a party. Many to many relationships have also been solved by implementing bridge-tables, and relationship arrows have been rotated to point to where the foreign key originated from.



5. Implementation

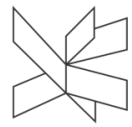
This section of the report showcases and explains the architectural patterns implemented in the software encompassing patterns such as MVVM, Client-Server, Producer-Consumer blocking, Adapter-Proxy, Singleton and Sockets. Also, through the following section it can be understood how all of these come together in playing a role in the Party Planner application, resulting in a stable multi-user system.

5.1) from GUI to Database and back

5.1.1) Overview

When a user interacts with the GUI and thus triggers a data request — for example loading the party list — it travels through six distinct layers before returning to the screen. The `Controller` reacts to user input and forwards immediately to the `ViewModel`, which is what the `View` is bound to through the Observer pattern. The `ViewModel` calls the `PartyClientModel`, the client-side Proxy that implements the same `PartyModel` interface as the server-side `ModelManager` — making the two indistinguishable from the `ViewModel`'s perspective. `PartyClientModel` serialises the request to JSON and sends it over a TCP socket to the `PartyClientHandler`, which runs on its own dedicated server thread. The handler delegates to `ModelManager`, which in turn calls the related `DAO` to query the PostgreSQL database. The response travels back as a JSON object, fires a `propertyChange` event on the `ViewModel`, and the UI updates through `Platform.runLater` — all without the `Controller` needing to do anything at all.

In the following paragraphs it can be seen how the **MyParties** view is filled by data specifically through this same process mentioned above.



5.1.2) GUI

```
<Pane maxHeight="-Infinity" maxWidth="-Infinity" minHeight="-Infinity" minWidth="-Infinity" prefHeight="528.0" prefWidth="837.0" stylesheet="
  <children>
    <Pane layoutX="-1.0" layoutY="-2.0" prefHeight="534.0" prefWidth="118.0" stylesheets="@../style.css">
      <children>
        <Label layoutX="14.0" layoutY="14.0" onMouseClicked="#onMyParties" prefHeight="26.0" prefWidth="88.0" styleClass="nav-label">
          <font>
            <Font size="20.0" />
          </font>
        </Label>
        <Label layoutX="14.0" layoutY="48.0" onMouseClicked="#onDiscover" prefHeight="25.0" prefWidth="82.0" styleClass="nav-label">
          <font>
            <Font size="14.0" />
          </font>
        </Label>
        <Label layoutX="14.0" layoutY="82.0" onMouseClicked="#onFriends" prefHeight="25.0" prefWidth="82.0" styleClass="nav-label">
          <font>
            <Font size="14.0" />
          </font>
        </Label>
        <Separator layoutX="123.0" layoutY="2.0" orientation="VERTICAL" prefHeight="528.0" prefWidth="0.0" />
        <Button fx:id="logoutButton" layoutX="28.0" layoutY="495.0" mnemonicParsing="false" onAction="#onLogout" text="log out" />
        <Label fx:id="userLabel" layoutX="7.0" layoutY="469.0" text="username" />
      </children>
    </Pane>
```





5.1.3) Controller

The `MyPartiesController` acts as a **coordinator** between the `MyParties.fxml` **FXML** view and its elements - and the `MyPartiesViewModel`, containing no business logic whatsoever.

```
public void init(ViewHandler viewHandler, @NotNull MyPartiesViewModel viewModel, Region root) { 1 usage 2 batorgabora +2
    this.root = root;
    this.viewModel = viewModel;
    this.viewHandler = viewHandler;

    errorLabel.textProperty().bind(viewModel.errorProperty());

    // bind list once
    partyList.setItems(viewModel.partiesProperty());

    partyList.setCellFactory( ListView<Party> lv -> updateItem(party, empty) -> { 3 batorgabora
        super.updateItem(party, empty);
        if (empty || party == null) { setText(null); setStyle(""); return; }
        boolean isOrganizer = party.getOrganizer() != null &&
            party.getOrganizer().getId().equals(LocalUser.getUser().getId());
        setText(party.getName() + (isOrganizer ? " (organizer)" : ""));
        setStyle(isOrganizer ? "-fx-font-weight: bold;" : "");
    });

    partyList.getSelectionModel().selectedItemProperty().addListener(
        ( ObservableValue<extends Party> obs, Party oldVal, Party newVal) -> viewModel.selectedPartyProperty().set(newVal));

    partyList.getSelectionModel().selectedItemProperty().addListener(
        ( ObservableValue<extends Party> obs, Party oldVal, Party newVal) -> selectedLabel.setText(newVal != null ? newVal.getName() : ""));

    loadParties();
}
```

In the `init` method, the controller establishes all necessary **bindings** once and never interacts with the data directly again.

Whenever the observable list is updated in the `ViewModel`, JavaFX automatically re-renders the `ListView` without any intervention from the controller, also, any error coming through the Observer pattern instantly appears in the view.

Two selection listeners are also added to wire the user's selection back to the shared `selectedPartyProperty` and update the `selectedLabel`.



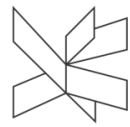
```
private void loadParties() { 2 usages  batorgabora
    userLabel.setText(LocalUser.getUser().getUsername());
    partyList.setVisible(false);
    selectedLabel.setVisible(false);
    createButton.setVisible(false);
    furtherButton.setVisible(false);
    loadingIndicator.setVisible(true);

    new Thread() -> {
        viewModel.updateParties();
        Platform.runLater(() -> {
            partyList.setVisible(true);
            selectedLabel.setVisible(true);
            createButton.setVisible(true);
            furtherButton.setVisible(true);
            loadingIndicator.setVisible(false);
        });
    }).start();
}
```

After these bindings are established, `loadParties()` is called, which hides the list and the buttons, shows the spinning loading indicator (the cat), and creates a background thread that calls `viewModel.updateParties()` — this way keeping the UI responsive during the network call.

Once the data arrives and the observable list is updated by the ViewModel, the controller simply makes the content visible again.

From this point forward the controller is entirely passive — navigation methods like `onFurther()` and `onCreateParty()` simply call `viewHandler.openView()` with no data handling involved.



5.1.4) ViewModel

The `MyPartiesViewModel` is the central piece of the **Observer pattern** implementation.

```
public class MyPartiesViewModel implements PropertyChangeListener { 7 usages  ⚡ batorgabora +2

    private final PartyService model; 2 usages
    private final ObjectProperty<Party> selectedParty; 3 usages
    private final StringProperty error = new SimpleStringProperty( initialValue: "" ); 2 usages
    private final ObservableList<Party> parties = FXCollections.observableArrayList(); 3 usages
    private final Gson gson = new Gson(); 1 usage

    public MyPartiesViewModel( @NotNull PartyService model, ObjectProperty<Party> selectedParty ) { 1 usage  ⚡ batorgabora +1
        this.model = model;
        this.selectedParty = selectedParty;
        model.addListener( propertyName: "getMyParties", listener: this );
        model.addListener( propertyName: "error", listener: this );
    }

    @Override public void propertyChange( @NotNull PropertyChangeEvent evt ) { ⚡ batorgabora
        switch ( evt.getPropertyName() ) {
            case "error" -> Platform.runLater( () ->
                error.set( (String) evt.getNewValue() );
            case "getMyParties" -> {
                JsonObject json = (JsonObject) evt.getNewValue();
                Type type = new TypeToken<List<Party>>(){}.getType(); ⚡ batorgabora
                List<Party> result = gson.fromJson( json.get( "data" ), type );
                Platform.runLater( () -> parties.setAll( result );
            }
        }
    }

    public void updateParties() { 1 usage ⚡ batorgabora
        var result = model.getMyParties( LocalUser.getUser() );
        Platform.runLater( () -> parties.setAll( result != null ? result : List.of() );
    }
}
```

What the View binds to:

- An `ObservableList<Party>` called `parties`,
- A `StringProperty` for errors, a
- A shared `ObjectProperty<Party>` for the selected party that is passed in from the `ViewModelFactory` and shared across ViewModels so navigation between views keeps the selection.

In the constructor, the ViewModel registers itself as a `PropertyChangeListener` on the model for two events:



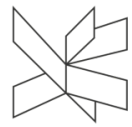
- `"getMyParties"`
- `"error"`

This is the Observer pattern in action — from this point forward the ViewModel reacts to the model instead of requesting data from it.

The `propertyChange` method handles both cases:

- When an `"error"` event is fired, it sets the `error` property on the `JavaFX` thread through `Platform.runLater`.
- When a `"getMyParties"` event is fired — either from a direct response or a server broadcast — the ViewModel extracts the `JsonObject` from the event, deserializes the `data` field into a `List<Party>` using `Gson's TypeToken`, and calls `parties.setAll(result)` on the `JavaFX` thread. Since the View's `partyList` is already bound to `parties`, the `ListView` updates automatically without the controller needing to do anything.

The `updateParties` method is the initial trigger — called once on load from the controller's background thread. It calls `model.getMyParties()` directly and sets the result, after which all further updates come exclusively through `propertyChange` via the implemented Observer pattern.



5.1.5) Model implementations

5.1.5.1) PartyModel (interface)

The `PartyModel` interface defines `getMyParties(User user)` as the **contract** that both sides must fulfill, enabling the **Proxy pattern** by ensuring that `PartyClientModel` and `ModelManager` are indistinguishable from the perspective of any class that depends on them.

```
ArrayList<Party> getMyParties(User user); 2 usages 2 implementations tonuvictor98-cpu
```

From the `ViewModel`'s perspective, it simply just calls the model's `getMyParties(LocalUser.getUser())` — it has no knowledge of what lies behind that call whatsoever.

5.1.5.2) ModelManager

The `ModelManager` is the real implementation of the `PartyModel` interface, started once in `ServerMain` and passed into every `PartyClientHandler` via the `Server`.

It is the **server-side "brain"** of the application — it knows nothing about sockets or JSON, only about the domain and the database.

```
@Override public ArrayList<Party> getMyParties(User user) { 2 usages batorgabora
    if (user == null) return new ArrayList<>();
    return new PartyDAO().getAcceptedByUser(user.getId());
}
```

When `model.getMyParties(user)` or any other method is called (by a `PartyClientHandler` method), `ModelManager` performs a simple null check and forwards immediately to `new PartyDAO().getAcceptedByUser(userId)`.

This is the **DAO pattern** — `ModelManager` never writes SQL directly (Oracle, n.d.) . Instead, it instantiates the appropriate DAO for each operation, keeping database



concerns cleanly separated from business logic. More on this can be found in the Database and DAO section of this chapter.

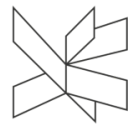
```
@Override public List<Item> addItem( @NotNull Party party, String name) { 2 usages  ⚙ batorgabora
    String id = "item-" + UUID.randomUUID().toString().substring(0, 8);
    new ItemDAO().create(id, name, quantity: 1, party.getId());
    return getItems(party);
}
```

For operations that change/mutate already existing data the flow is slightly richer. When `addItem` is called for example, `ModelManager` generates a **UUID** (used as primary key in the database) for the new item and then calls `ItemDAO.create()` to update the database and then immediately calls `getItems()` and returns the full updated list.

This is what allows the server to respond with fresh data in a single round trip — the client receives the updated data directly without needing a separate request for it.

5.1.5.3) PartyClientModel

The `PartyClientModel` is the **client-side Proxy implementation** of `PartyModel`, bridging the gap between the `ViewModel` layer and the server through a **TCP socket** connection.



```
public PartyClientModel(String host, int port) { 1 usage  ⚡ batorgabora +1
    this.client = new PartyClient(host, port);
    this.gson = new Gson();

    Thread listenerThread = new Thread() -> {
        while (true) {
            String raw = client.receive();
            if (raw != null) {
                try {
                    JSONObject json = JsonParser.parseString(raw).getAsJsonObject();
                    String msgType = json.has( memberName: "type" ) ? json.get("type").getString() : "";

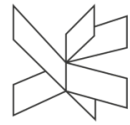
                    if ("error".equals(msgType)) {
                        // errors go through Observer and also unblock sendAndReceive
                        support.firePropertyChange( propertyName: "error", oldValue: null,
                            json.has( memberName: "message" ) ? json.get("message").getString() : "unknown error");
                        responseQueue.put(json);
                    } else if ("broadcast".equals(msgType)) {
                        // broadcasts only go through Observer — no sendAndReceive is waiting
                        if (json.has( memberName: "action" )) {
                            support.firePropertyChange(json.get("action").getString(), oldValue: null, json);
                        }
                    } else {
                        // direct responses only go in the queue — sendAndReceive handles the rest
                        responseQueue.put(json);
                    }
                } catch (Exception e) {
                    System.out.println("Listener error: " + e.getMessage());
                }
            }
        }
    };
    listenerThread.setDaemon(true);
    listenerThread.start();
}
```

```
private synchronized @NotNull JSONObject sendAndReceive( @NotNull Runnable sendAction) { 37 usages  ⚡ batorgabora
    try {
        sendAction.run();
        return responseQueue.take();
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
        throw new IllegalStateException("Interrupted waiting for response");
    }
}
```

```
public String receive() 1 usage  ⚡ batorgabora
{
    try { return inputReader.readLine(); }
    catch (IOException e) { throw new IllegalStateException("Connection to server was lost.", e);}
}
```

In the constructor, a **daemon thread** — a background thread that automatically dies when all other threads finish — is started and runs an infinite loop, continuously calling `client.receive()` which blocks on `inputReader.readLine()` waiting for data from the server.

When a message arrives, the listener thread determines its type and handles it



accordingly — this is the point where the request-response mechanism and the **Observer pattern** diverge into two clean separate responsibilities.

1. If the message type is "**broadcast**" — meaning it was sent by the server as a push notification triggered by another client's action — it is handled exclusively through the **Observer pattern** as `support.firePropertyChange` is called with the action name and the full `JsonObject` as the new value, notifying any registered `PropertyChangeListener`. The relevant `ViewModel` reacts in its `propertyChange` method and updates its observable properties on the `JavaFX` thread via `Platform.runLater`. No `sendAndReceive` is waiting and the queue is never touched — this is pure Observer-driven push.
2. If the message type is "**response**" — a direct reply to this client's own request — it is placed into the `responseQueue`, which unblocks the `sendAndReceive` call waiting on `responseQueue.take()`. This is the **Consumer** receiving what the **Producer** just placed in the queue, completing the request-response cycle and returning the result to the calling thread.

If the response type is "**error**", the listener fires `propertyChange("error")` to forward the error to the `ViewModel` and also puts the **JSON** in the queue to unblock `sendAndReceive`, since the calling thread always has to be released regardless of success or failure.

Example method:

```
@Override public ArrayList<Party> getMyParties(User user) { 2 usages  ⚡ batorgabora
    JsonObject json = sendAndReceive(() -> client.requestGetMyParties(user.getId()));
    if (isError(json)) return new ArrayList<>();
    Type type = new TypeToken<ArrayList<Party>>().getType();  ⚡ batorgabora
    return gson.fromJson(json.get("data"), type);
}
```

`getMyParties` is a clear example of the **Proxy pattern** in action. The method signature is identical to `ModelManager`'s implementation — the `ViewModel` cannot distinguish

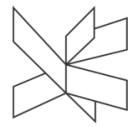


between them. Internally it forwards to `sendAndReceive`, passing a **lambda** — a concise anonymous function passed as an argument — that calls `client.requestGetMyParties(user.getId())`, which constructs and sends the **JSON** request over **TCP**.

As `sendAndReceive` is **synchronized**, it is ensured that only one request-response cycle is in progress at a time, and blocks on `responseQueue.take()` until the listener thread places the server's response in the queue. Once unblocked, `Gson`'s `TypeToken` is used to deserialize the datafield back into an `ArrayList<Party>`, which is necessary because Java's type erasure means generic type information must be explicitly preserved at runtime. The result is returned to the `ViewModel` exactly as `ModelManager` would return it from a direct database query, completing the transparent proxy illusion

5.1.5.4) To conclude:

The key insight is that both implementations return the same type and honour the same contract — the `ViewModel` cannot distinguish between them. This **decoupling** means the client code is completely independent of the network layer, and in theory the entire client-server communication could be replaced without touching a single `ViewModel` or `Controller`.



5.1.6) PartyClientHandler

The `PartyClientHandler` is the **server-side counterpart** to `PartyClientModel`, running on its own thread for each connected client.

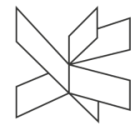
```
public PartyClientHandler( @NotNull Socket socket, PartyModel model, List<PartyClientHandler> handlers) { 1 usage 2 batorgabora +1
    this.handlers = handlers;
    this.socket = socket;
    this.model = model;
    this.gson = new Gson();
    try {
        inputReader = new BufferedReader(new InputStreamReader(socket.getInputStream(), StandardCharsets.UTF_8));
        outputWriter = new PrintWriter(new OutputStreamWriter(socket.getOutputStream(), StandardCharsets.UTF_8), autoFlush: true);
    } catch (IOException e) {
        throw new IllegalStateException("Could not initialize client handler.", e);
    }
    registerHandlers();
}

private void registerHandlers() { 1 usage 2 tonuvictor98-cpu
    dispatch.put(Action.LOGIN, this::handleLogin);
    dispatch.put(Action.CREATE_ACCOUNT, this::handleCreateAccount);
    dispatch.put(Action.GET_ALL_USERS, JsonObject req -> handleGetAllUsers());
    dispatch.put(Action.ADD_FRIEND, this::handleAddFriend);
    dispatch.put(Action.GET_ALL, this::handleGetAll);
    dispatch.put(Action.GET_MY_PARTIES, this::handleGetMyParties);
    dispatch.put(Action.GET_INVITED_PARTIES, this::handleGetInvitedParties);
}
```

```
private void close() { 1 usage 2 batorgabora
    handlers.remove(0: this);
    try { socket.close(); } catch (IOException ignored) {}
}

@Override public void run() { 2 batorgabora
    try {
        String line;
        while ((line = inputReader.readLine()) != null) {
            JsonObject request = JsonParser.parseString(line).getAsJsonObject();
            handleRequest(request);
        }
    } catch (IOException e) {
        System.out.println("Client disconnected.");
    } finally {
        close();
    }
}
```

When a request arrives over the **TCP socket**, the `run()` loop reads it line by line, parses it into a `JsonObject`, and passes it to `handleRequest`, which uses `Action.fromValue()` to map the action string to an `Action` enum constant and route it to the correct handler method via a switch statement.



```
private void handleGetMyParties( @NotNull JsonObject request) { 1 usage  ⚡ batorgabora +1
    User user = new UserDAO().getId(request.get("userId").getString());
    if (user == null) { sendResponse( action: "getMyParties", data: ""); return; }
    sendResponse( action: "getMyParties", gson.toJson(model.getMyParties(user)));
}
```

For `getMyParties`, the handler extracts the `userId` from the request, fetches the corresponding `User` object via `UserDAO`, and calls `model.getMyParties(user)` on the `ModelManager`—the real implementation.

The result is serialized to **JSON** using `Gson` and sent back to the client via `sendResponse`, which wraps it in a structured response object containing the action name and data field that `PartyClientModel`'s listener thread expects on the other end.

```
private void sendBroadcast(String action, String data) { 1 usage  ⚡ batorgabora
    JsonObject response = new JsonObject();
    response.addProperty( property: "type", value: "broadcast");
    response.addProperty( property: "action", action);
    try {
        response.add( property: "data", JsonParser.parseString(data));
    } catch (Exception e) {
        response.addProperty( property: "data", data);
    }
    outputWriter.println(gson.toJson(response));
}

private void broadcastToOthers(String action, String data) { 12 usages  ⚡ batorgabora +1
    synchronized (handlers) {
        for (PartyClientHandler handler : handlers) {
            if (handler != this) {
                handler.sendBroadcast(action, data);
            }
        }
    }
}
```

For **mutation operations** such as `addItem` or `voteForOption`, the handler goes one step further — after sending the response to the requesting client, it calls `broadcastToOthers`, which iterates the shared handlers list held by the `Server` and sends the same updated data to every other connected client. This shared list is a `Collections.synchronizedList` to ensure thread safety when multiple handlers write to it simultaneously. This is what enables real-time multi-user synchronization — when one user adds an item, all other users viewing the same party see the update immediately without refreshing



5.1.7) Server – PartyClient

5.1.7.1) Server:

The `Server` class is the **entry point for all client connections**.

```
public class Server { 3 usages 2 batorgabora
    public static final int PORT = 9999; 2 usages
    private final PartyModel model; 2 usages
    private final List<PartyClientHandler> handlers = Collections.synchronizedList(new ArrayList<>()); 2 usages

    public Server(PartyModel model) { this.model = model; }

    public void start() { 2 batorgabora
        try (ServerSocket serverSocket = new ServerSocket(PORT)) {
            System.out.println("Server started on port " + PORT);
            while (true) {
                Socket socket = serverSocket.accept();
                System.out.println("Client connected: " + socket.getInetAddress());
                PartyClientHandler handler = new PartyClientHandler(socket, model, handlers);
                handlers.add(handler);
                new Thread(handler).start();
            }
        } catch (IOException e) {
            throw new IllegalStateException("Server failed.", e);
        }
    }
}
```

It opens a `ServerSocket` on **port 9999** and enters an infinite loop, blocking on `serverSocket.accept()` until a client connects. When a connection arrives, it instantiates a new `PartyClientHandler` for that socket and starts it on its own thread — meaning each client gets a dedicated handler thread for the lifetime of its connection. The shared handlers list — a `Collections.synchronizedList` — is passed to every handler on construction, enabling broadcast communication between clients. When a client disconnects, the handler removes itself from this list in `close()`, ensuring broadcasts are never sent to dead connections.



5.1.7.2) PartyClient:

`PartyClient` is the **client-side transport layer** — it knows nothing about the application protocol, only the connection to the server and the handling of incoming data.

```
public PartyClient(String host, int port) 1 usage 2 batorgabora
{
    try {
        socket = new Socket(host, port);
        inputReader = new BufferedReader(
            new InputStreamReader(socket.getInputStream(), StandardCharsets.UTF_8));
        outputWriter = new PrintWriter(
            new OutputStreamWriter(socket.getOutputStream(), StandardCharsets.UTF_8),
            autoFlush: true);
        gson = new Gson();
    }
    catch (IOException e) {
        throw new IllegalStateException("Could not connect to server.", e);
    }
}

public String receive() 1 usage 2 batorgabora
{
    try { return inputReader.readLine(); }
    catch (IOException e) { throw new IllegalStateException("Connection to server was lost.", e); }
}
```

In the constructor it establishes a **Socket connection** to the server, wrapping the streams in a `BufferedReader` for reading and a `PrintWriter` with auto-flush for writing.

```
private synchronized void sendRequest(JsonObject request) 37 usages 2 batorgabora
{
    outputWriter.println(gson.toJson(request));
}
```

The `sendRequest` method is **synchronized** to prevent two threads writing to the socket simultaneously — without this, two concurrent requests could both try to forward their **JSON** on the wire and corrupt the stream.

```
public void requestGetMyParties(String userId) { 1 usage 2 batorgabora
    JsonObject request = createRequest(Action.GET_MY_PARTIES);
    request.addProperty("userId", userId);
    sendRequest(request);
}
```

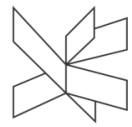


Every request `requestGetMyParties`, `requestAddItem`, `requestVoteForOption` and so on — constructs a `JsonObject` with the appropriate action string and parameters and calls `sendRequest`.

```
public String receive() 1 usage 2 batorgabora
{
    try { return inputReader.readLine(); }
    catch (IOException e) { throw new IllegalStateException("Connection to server was lost.", e);}
}
```

The `receive` method simply calls `inputReader.readLine()`, blocking until a complete line arrives.

All protocol interpretation — what the **JSON** means, what to do with the response — happens in the `PartyClientModel` above it. `PartyClient` is purely the **transport**, keeping network I/O concerns separate from application logic.



5.1.8) Database Connection

The `DataBaseConnection` class is the server's single point of access to the **Neon PostgreSQL** database.

```
public class DataBaseConnection { 73 usages  ⚡ batorgabora +1
    private static final String URL = "jdbc:postgresql://ep-solitary-sunset-alixchh4-pooler.c-3.eu-central-1.aws.neon.tech/party_planner?sslm
    private static final String USER = "neondb_owner"; 1 usage
    private static final String PASSWORD = "npg_A6rYzvCW5Day"; 1 usage

    private static final DataBaseConnection instance = new DataBaseConnection(); 1 usage

    // each thread gets its own connection — no sharing between threads
    private static final ThreadLocal<Connection> threadLocalConnection = new ThreadLocal<>(); 2 usages

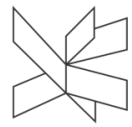
    @Contract(pure = true)
    private DataBaseConnection() {} 1 usage  ⚡ batorgabora

    @Contract(pure = true)
    public static DataBaseConnection getInstance() { 61 usages  ⚡ batorgabora +1
        return instance;
    }

    public Connection getConnection() throws SQLException { ⚡ batorgabora
        Connection conn = threadLocalConnection.get();
        if (conn == null || conn.isClosed()) {
            conn = DriverManager.getConnection(URL, USER, PASSWORD);
            conn.createStatement().execute( sql: "SET search_path TO party_planner");
            threadLocalConnection.set(conn);
        }
        return conn;
    }
}
```

`DataBaseConnection` follows the **Singleton pattern** — a single instance is created once and accessed globally via the static `getInstance()` method throughout the entire (server side) application.

Rather than storing a single shared `Connection` object which would cause conditions under simultaneous load, it uses a `ThreadLocal<Connection>`. This means each server thread — one per connected client — receives its own dedicated database connection the very first time it calls `getConnection()`. If the connection is null or has been closed, a new one is opened to the **Neon PostgreSQL** database using the **JDBC*** driver (The PostgreSQL Global Development Group, n.d.) with **SSL**** required. The connection is then cached in the `ThreadLocal` and reused for all subsequent queries on that thread. This approach combines the controlled usage of the **Singleton pattern** with thread safety,



eliminating the **I/O errorfailures***** that could occur under concurrent load when multiple clients interact simultaneously and compete over a shared connection.

* Java Database Connectivity, the standard Java API for connecting to relational databases

** Secure Sockets Layer, encrypts the connection between the server and the database so credentials and query data cannot be intercepted in transit

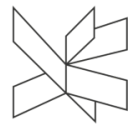
*** Input/Output error, thrown when a thread tries to use a database connection that another thread is simultaneously using or has already closed.

5.1.9) PartyDAO

PartyDAO is responsible for all **SQL operations** related to the party table in the party_planner schema.

```
public ArrayList<Party> getAcceptedByUser(String userid) {
    String sql = "SELECT p.* FROM party_planner.party p JOIN party_planner.partyusers pu ON p.partyid = pu.partyid " +
        "WHERE pu.userid = ? AND (pu.status = 'accepted' OR pu.role = 'organizer')";
    ArrayList<Party> parties = new ArrayList<>();
    try (Connection conn = DataBaseConnection.getInstance().getConnection();
        PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setString(1, userid);
        ResultSet rs = ps.executeQuery();
        while (rs.next()) {
            Party p = mapRow(rs);
            System.out.println("Loaded accepted party: " + p.getName());
            parties.add(p);
        }
    } catch (SQLException e) {
        throw new RuntimeException(e);
    }
    return parties;
}
```

When ModelManager calls getAcceptedByUser(userId), PartyDAO executes a JOIN between party_planner.party and party_planner.partyusers, filtering for rows where the user's status is "accepted" or their role is "organizer" — covering both participants who accepted an invitation and the organizer of the party



```
private @NotNull Party mapRow( @NotNull ResultSet rs) throws SQLException { 4 usages  ⚡ batorgabora
    String partyId = rs.getString( columnLabel: "partyid");
    String name = rs.getString( columnLabel: "name");
    String description = rs.getString( columnLabel: "description");
    String location = rs.getString( columnLabel: "location");
    LocalDate date = rs.getDate( columnLabel: "date") != null ? rs.getDate( columnLabel: "date").toLocalDate() : null;
    User organizer = getOrganizerForParty(partyId);
    return new Party(partyId, name, description, location, date, organizer);
}

private @Nullable User getOrganizerForParty(String partyId) { 1 usage  ⚡ batorgabora +2
    String sql = "SELECT userid FROM party_planner.partyusers WHERE partyid = ? AND role = 'organizer'";
    try (Connection conn = DataBaseConnection.getInstance().getConnection();
        PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setString( parameterIndex: 1, partyId);
        ResultSet rs = ps.executeQuery();
        if (rs.next()) return new UserDAO().getById(rs.getString( columnLabel: "userid"));
    } catch (SQLException e) {
        log.severe( msg: "getOrganizerForParty failed for partyId=" + partyId + ": " + e.getMessage());
    }
    return null;
}
}
```

Each row is formed into a Party domain object via `mapRow`, which additionally calls `getOrganizerForParty` to fetch the organizer's User object — a second query per party that populates the organizer field used later by the various views to determine whether to show the edit button and to highlight specific parties in the my parties page...

All queries use `Connection` and `PreparedStatement`, ensuring connections are returned cleanly after each operation.

```
public void delete(String partyid) { 2 tonuvictor98-cpu +2
    try (Connection conn = DataBaseConnection.getInstance().getConnection()) {
        try (PreparedStatement ps = conn.prepareStatement(
            sql: "DELETE FROM party_planner.voteoption WHERE optionid IN (SELECT optionid FROM party_planner.option WHERE partyid = ?)")) {
            ps.setString( parameterIndex: 1, partyid);
            ps.executeUpdate();
        }
        try (PreparedStatement ps = conn.prepareStatement(
            sql: "DELETE FROM party_planner.option WHERE partyid = ?")) {
            ps.setString( parameterIndex: 1, partyid);
            ps.executeUpdate();
        }
        try (PreparedStatement ps = conn.prepareStatement(
            sql: "DELETE FROM party_planner.claimitem WHERE itemid IN (SELECT itemid FROM party_planner.item WHERE partyid = ?)")) {
            ps.setString( parameterIndex: 1, partyid);
            ps.executeUpdate();
        }
        try (PreparedStatement ps = conn.prepareStatement(
            sql: "DELETE FROM party_planner.item WHERE partyid = ?")) {
            ps.setString( parameterIndex: 1, partyid);
            ps.executeUpdate();
        }
    }
}
```



The `delete` method may also be worth highlighting as it performs a **cascading cleanup** — deleting `votes`, `options`, `claims`, `items`, and `partyusers` entries before finally deleting the `party` itself, making sure no “ghost data” / non-existent references remain, without having to rely on database-level cascades which are hard to keep track of.

5.1.10) Party model class

```
public Party(String id, String name, String description, String location, LocalDate date, User organizer) {
    this.id = id;
    this.name = name;
    this.description = description;
    this.location = location;
    this.date = date != null ? date.toString() : null;
    this.organizer = organizer;
    this.participants = new ArrayList<>();
    this.itemList = new ItemList();
}
```

The `Party` class is the **central domain object** of the application, representing a party with all its associated data — `name`, `description`, `location`, `date`, `organizer` and `participants` — and is passed between all layers of the system from the database through the server to the client.

5.1.11) Threading across the application

Thread safety is a recurring concern throughout the system and several conscious decisions were made to address it at each layer.

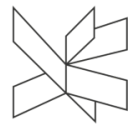
On the server, `ThreadLocal<Connection>` in `DataBaseConnection` gives each handler thread its own database connection, eliminating the problem of multiple threads competing over and corrupting a single shared connection, while `Collections.synchronizedList` on the shared handlers list ensures that adding, removing and iterating over handlers from different threads simultaneously does not produce unpredictable or incorrect results.



On the client, `synchronized sendRequest` in `PartyClient` ensures only one thread writes to the socket at a time, and the `synchronized sendAndReceive` guarantees only one request-response cycle is happening at any moment — preventing threads from stealing each other's responses from the shared queue.

Finally, all `ViewModel` updates reach the UI via `Platform.runLater`, respecting JavaFX's single-threaded rendering model.

Together these decisions form a coherent thread safety strategy across every layer of the system.



6. Testing

This section of the report dives into the tests carried out in order to confirm the reliability and user operability of the system. And if the use cases are implemented and functioning. The create party use case and its testing infrastructure will be detailed below and how the **ZOMBIE** model was applied (Grenning, n.d.) . For these tests both white box black box and even integration tests were utilized and detailed below.

6.1) Unit testing

The purpose and goal of unit testing is to verify one part of the systems functionality in isolation which streamlines the process of checking if recent code changes affect functionality, thus instead of having to manually run through every test scenario after a session of development one must simply run the pre built tests,

Given that Parties are at the core of this application the `createParty` functionality was chosen as the target of the white box tests.

```
@BeforeEach void setUp() {  @ tonuvictor98-cpu
    model = Mockito.mock(PartyModel.class);
    fakeUser = new User( username: "micheal jackson", password: "hee hee", mail: "test@test.com");
    LocalUser.setUser(fakeUser);
    vm = new CreatePartyViewModel(model);
}
```

Before each test a mock class is set up using `mockito` (Mockito, n.d.) , for the party creation features to work, a `User` must be logged in hence the `LocalUser` and a new `ViewModel`, this is done to avoid interference between tests



Z - zero

The first letter of the zombies testing methodology stands for zero. It requires checking how the system deals with null or empty queries

```
@Test void emptyFields_blocked() {  @tonuvictor98-cpu *
    vm.createParty();

    assertFalse(vm.errorProperty().get().isEmpty());
    verify(model, never()).createParty(any(), any(), any(), any(), any());
}
```

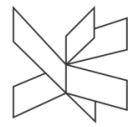
A simple test which initiates the `ViewModel createParty` method with no data. The method returns true when the creation was successful and false when it wasn't, thus it asserts that it should return false as a condition for the tests passing, for the second condition the code snippet verifies that the models own `createParty` method was not called which ensures that blank parties are not sent forward to the `model` at all.

```
@Test void missingDate_blocked() {  @tonuvictor98-cpu *
    vm.nameProperty().set("doesn't");
    vm.locationProperty().set("matter");
    vm.dateProperty().set(null);

    vm.createParty();

    assertFalse(vm.errorProperty().get().isEmpty());
    verify(model, never()).createParty(any(), any(), any(), any(), any());
}
```

The same as the first code snippet with the caveat that the name and location of the party



are declared but the date isn't, the requirements for passing the test are the same as the system is expected to not forward this party

O - one

stands for testing the functionality with minimum input.

```
@Test void validInputs_correctArgsForwardedToModel() throws InterruptedException { ② tonuvictor98-cpu *
    LocalDate year = LocalDate.of( year: 2027, month: 1, dayOfMonth: 1);
    vm.nameProperty().set("eurovision");
    vm.locationProperty().set("moldova");
    vm.dateProperty().set(year);
    Party realFutureItWillHappen = new Party(UUID.randomUUID().toString(), name: "eurovision", description: "")

    when(model.createParty(any(), any(), any(), any(), any())).thenReturn(realFutureItWillHappen);
    vm.createParty();
    Thread.sleep( millis: 200);

    verify(model).createParty(eq( value: "eurovision"), eq( value: ""), eq( value: "moldova"), any(), eq(year));
    assertTrue(vm.errorProperty().get().isEmpty());
}
```

The fields a user enters when creating the party are artificially declared again, description is notably left out as it is not mandatory. When/then return is used to simulate an interaction with the server. The verification checks if the viewmodel considers the party created and the code verifies that the model got the exact data it should for the party creation.



M - many

Tests the systems ability to handle multiple queries simultaneously

```
@Test void calledTwice_modelReachedBothTimes() throws InterruptedException {
    LocalDate today = LocalDate.now();
    vm.nameProperty().set("Party One");
    vm.locationProperty().set("Oslo");
    vm.dateProperty().set(today);
    Party fakeParty = new Party(UUID.randomUUID().toString(), name: "Party One", description: "", location: "Oslo", to
    when(model.createParty(any(), any(), any(), any(), any())).thenReturn(fakeParty);

    vm.createParty();
    vm.createParty();
    Thread.sleep( millis: 200);

    verify(model, times( wantedNumberOfInvocations: 2)).createParty(eq( value: "Party One"), eq( value: ""), eq( value: "Oslo"
})
```

The same party creation testing however is fired twice in the viewmodel each time with the expectation of success, the code then checks that models create party method was called twice too and with the correct data.

B - boundary

Tests the boundary of the system



```
@Test void whitespaceOnlyName_blocked() {  @ tonuvictor98-cpu *
    vm.nameProperty().set(" ");
    vm.locationProperty().set("whatever room Loke finds");
    vm.dateProperty().set(LocalDate.now());

    vm.createParty();

    assertFalse(vm.errorProperty().get().isEmpty());
    verify(model, never()).createParty(any(), any(), any(), any(), any());
}

@Test void pastDate_blocked() {  @ tonuvictor98-cpu *
    vm.nameProperty().set("Time traveller party");
    vm.locationProperty().set("idk where that was");
    vm.dateProperty().set(LocalDate.of(year: 2000, month: 1, dayOfMonth: 1));

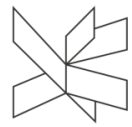
    vm.createParty();

    assertFalse(vm.errorProperty().get().isEmpty());
    verify(model, never()).createParty(any(), any(), any(), any(), any());
}
```

The system tests to ensure that a name property with empty space in it does not fire the party creation process, same goes for a date that has already passed.

E - exception

Testing for the unexpected exceptions



```
@Test void clearError_resetsErrorProperty() {  tor
    vm.errorProperty().set("kaboom");

    vm.clear();

    assertTrue(vm.errorProperty().get().isEmpty());
}
```

Checks if the error handling within the viewmodel by ensuring nothing interferes with the classes error properties.

S - simple

```
@Test void properties_reflectSetValues() {  tonuvictor98-cpu
    vm.nameProperty().set("a");
    vm.locationProperty().set("aa");
    vm.descriptionProperty().set("aaa");

    assertEquals(expected: "a", vm.nameProperty().get());
    assertEquals(expected: "aa", vm.locationProperty().get());
    assertEquals(expected: "aaa", vm.descriptionProperty().get());
}
```

Checks if the viewmodels setter and getter methods work.



6.2) Integration testing

In a system of this scale many sub systems must seamlessly communicate and coordinate in order to create a smooth user experience. As such the integration testing is vital to ensure the separate parts of the system are continuously connected.

```
@BeforeEach public void setUp() throws Exception {
    tonuvictor98-cpu
    {
        try { Platform.startup(() -> {}); } catch (IllegalStateException ignored) {}

        testUser = new User( id: "o", username: "ye", password: "pass", mail: "test@test.com");
        LocalUser.setUser(testUser);

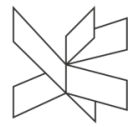
        mockModel = Mockito.mock(PartyModel.class);
        mockViewHandler = Mockito.mock(ViewHandler.class);
        viewModel = new CreatePartyViewModel(mockModel);
        controller = new CreatePartyController();

        Field vmField = CreatePartyController.class.getDeclaredField( name: "viewModel");
        vmField.setAccessible(true);
        vmField.set(controller, viewModel);

        Field vhField = CreatePartyController.class.getDeclaredField( name: "viewhandler");
        vhField.setAccessible(true);
        vhField.set(controller, mockViewHandler);

        viewModel.createdPartyProperty().addListener(( ObservableValue<extends Party> obs, Party o, Party n) -> {
            if (n != null) mockViewHandler.openView( id: "my parties");
        });
    }
}
```

The test dependencies are initiated with the exception of the controller which would require the javafx thread in order to be initialized which a test environment does not have access to, thus instead java reflection is used to retrieve the fields for viewmodel and viewhandler.



```
@Test public void validInput_navigatesToMyParties() throws InterruptedException {
    tonuvictor98-cpu
    {
        viewModel.nameProperty().set("bs");
        viewModel.locationProperty().set("bs place");
        viewModel.dateProperty().set(LocalDate.now());
        when(mockModel.createParty(any(), any(), any(), any(), any()))
            .thenReturn(new Party(UUID.randomUUID().toString(), name: "bs", description: "", location: "bs place", LocalDate.now(), testUser));

        controller.onCreate();
        Thread.sleep(200);
        CountDownLatch latch = new CountDownLatch(1);
        Platform.runLater(latch::countDown);
        latch.await(5, TimeUnit.SECONDS);

        verify(mockViewHandler).openView(id: "my parties");
    }
}
```

The party creation part of the test, mimics previous tests, the difference is that the controllers `onCreat()` method is also called which is bound to the create party button, meaning that the test expects the tab to change from party creation to my parties which it checks for by verifying if the `viewhandler` has opened that view since the beginning of the test.

```
@Test public void invalidInput_staysOnForm() {
    tonuvictor98-cpu
    {
        controller.onCreate();

        verify(mockViewHandler, never()).openView(any());
        verify(mockModel, never()).createParty(any(), any(), any(), any(), any());
    }
}
```

When a blank party is created the `viewhandler` shouldn't open anything and remain on the party creation page and the party creation methods in `viewmodel` should never be called, the test checks for that with `verify`

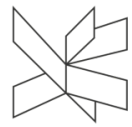


```
@Test public void serverError_staysOnForm() throws InterruptedException {
    tonuvictor98-cpu
    {
        viewModel.nameProperty().set("i am tired mastah");
        viewModel.locationProperty().set("cotton fields");
        viewModel.dateProperty().set(LocalDate.of(year: 2027, month: 6, dayOfMonth: 15));
        when(mockModel.createParty(any(), any(), any(), any(), any())).thenReturn(t: null);

        controller.onCreate();
        Thread.sleep(millis: 200);

        verify(mockViewHandler, never()).openView(any());
        verify(mockModel).createParty(any(), any(), any(), any(), any());
    }
}
```

A normal party is created, however when it is used to simulate server side rejection then the test checks that in such a case the view does not change, it also checks that the `onCreate()` method fires the create party method in the model once.



6.3) System test

Blackbox testing scenarios table for the create party use case where every step of the party creation is tested and almost every way it could go wrong is checked for.

Step	Test	Passed
1	The user presses the create party button	✓
2	A party creation form appears with fields for name description location and date	✓
3	Users presses the create party button without filling in any of the fields and the system throws a party cannot be created error on screen	✓
4	User fills in name.	✓
5	User fills in description.	✓
6	User fills in location.	✓
7	User fills in date by typing it or choosing it from the date picker.	✓
8	User presses the create party button	✓
9	[if date is not a valid date format] System displays an error	✓
10	[if the name is not filled in] System displays an error	✓
11	User is taken to the my parties list where the party he has just created is displayed in a list	✓
12	The user presses the back to my parties button at any point during the party creation and is taken back to the previous page	✓

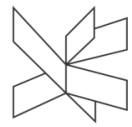
Table 6.1: Test scenarios: Create a party- PASSED



6.4) Testing summary

This chapter aimed to ensure the system is ready for real world contact with users by ensuring all the test cases based on use cases were checked for weaknesses and potential failure points. A plethora of tools and methodologies were used for this aim and described in detail here. To conclude the tests a table of the use cases and their current implementation is attached bellow

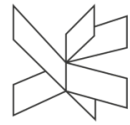
Use case	Result	Comment
Create Account	Mostly Working	Account creating functions as intended with checks for non identical usernames and proper email formatting as well as checking if the two passwords match. However there's no requirements for the passwords you can have a one character password, and password security measures like text symbol number mixing enforcement
Remove participant	Working	Can remove participants as the organizer, cannot remove yourself
Add participant	Working	Can add people from your friends list to a party
Create party	Working	Refer to the test scenarios above
Manage party	Working	Party details can be changed with no issues
Join party	Working	User can accept invitations on the discover tab
Leave party	Working	Participant can leave any party they are in
Manage item list	Working	Organizer can add and remove items
Mark item	Working	Any participant can claim an item
Choose date	Working	The organizer can add and remove date voting options



Vote date	Working	Participants can vote for the date options which updates the vote count and displays the top option above the rest
Chat	Working	Every party has its own chat in which member and organizer can send messages which appear in the chat of any other participant
Add friend	Working	Users can add any other user as friends
Edit account	Discarded	User can change username password or email

Table 6.2: Use cases black box testing overview

All of the systems envisioned functionality has been thoroughly tested and All of the core features are fully functional.



7. Results

The results will be reflected on, based on the integration of the requirements.

Functional requirements

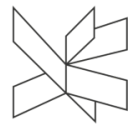
Nr.	Functional requirement	Yes/no	Note
1	As a user, I want to be able to create a username and password, so that I can have a presence on the app.	Yes	
2	As a user, I want to be able to log in to my account, so I can have a personalized experience.	Yes	
3	As an organizer, I want to create a party, so I can structure the event for several participants.	Yes	
4	As an organizer, I want to add participants to the event, so they can see the event.	Yes	
5	As a participant, I want to see the parties I'm invited to and join them, so I can interact with the party organization features.	Yes	
6	As an organizer, I want to be able to edit the party, to manage the	Yes	



	item list and the description of the event.		
7	As an organizer, I want to be able to set a list of needed items to a party I've created, so that the participants can help me bring the items needed.	Yes	
8	As a participant, I want to be able to mark off the items I would bring, so that the organizer and other participants don't bring duplicates.	Yes	
9	As an organizer, I want to be able to choose dates for the participants to vote for, so that everyone has a say on the date of the party.	Yes	
10	As a participant, I want to be able to vote for my preferred date of a party, so I can have a say in the planning.	Yes	
11	As a participant, I want to be able to leave parties I have already joined, as if something comes up this way my intentions are clear.	Yes	
12	As an organizer, I want to remove participants from the event, so I can manage the amount of participants.	Yes	



13	As a participant, I want to be able to use the party chat, so I can communicate with the rest of the party.	Yes	
14	As a user, I want to be able to add other users as friends, so that I can more easily interact with them through the system.	Yes	They can't interact with them through private chat, but when creating a party, you can invite users from a friends list.
15	As a user, I want to be able to remove a friend, since too many friends can get overwhelming	Yes	
16	As a participant and organizer, I want to receive a random small quest at the start of the party, so the party will have entertainment.	NO	This requirement did not get implemented and was seen as not important
17	As a user I want to be able to change my username, since I might want to change it in the future.	NO	This requirement did not get implemented since it was deemed as less important, and just an additional feature



Non-Functional requirements

Nr.	Non-Functional requirements	Yes/No	Note
1	The system shall implement a multi-user system.	Yes	
2	The system shall follow the MVVM work pattern.	Yes	
3	The database shall be made with SQL.	Yes	
4	The system shall be implemented in Java.	Yes	

Table 7.1: Use cases black box testing overview



8. Discussion

The results of the project indicate that the project was a success, 15 out of 17 functional requirements were met, while all non-functional requirements were followed. The only functional requirements that were not met, were considered of lesser relevance.

One of the omitted requirements was "As a user I want to be able to change my username, since I might want to change it in the future.". This requirement was not implemented in the system, but it was considered when creating the design of the system. Therefore, the database was designed to have the candidate key "username", but also the surrogate key "userId" to be used as a stable primary identifier.

The project also seems to satisfy the problem statement: "How can an organizer and participants work together to share the burden of event creation among themselves and streamline the management of an event?". An argument could be made that by implementing features where the participants of the party can help choose dates and bring items, the burden of being the organizer may be reduced.

The application is structured around a simple graphical interface that the user can easily navigate while keeping all of the features of planning an event. This structure is what an organizer needs, when planning an event. It keeps every detail in one place, instead of having to write private messages around, asking participants to bring certain items, asking what date they are free while having to create a party in an app like Facebook or CHECKIN. The Party Planner app centralizes these features, which may help relieve the stress placed on the organizer.

The results being successful could have been expected since the system was developed around a set of use cases. The chosen design patterns like MVVM observer and singleton appear to have been a good choice of design patterns specifically since the project requires that "The system shall implement a multi-user system.". The patterns structured



the system by separating responsibilities, supporting communication between components and managing the database connection.

The system was partially tested using the **ZOMBIES** testing approach (Grenning, n.d.) . The "Create party" use case was tested and the system passed. This provides evidence that one of the core functions of the system works as intended.

The rest of the other use cases remain only tested by **Blackbox** testing. Blackbox testing is not as thorough as ZOMBIES testing. Therefore, it's not confirmed that the system, in its entirety, is fully functional under the hood. Which for future projects is something that could be improved by testing the rest of the use cases to ensure reliability of the system.

In retrospect, the system could have been developed based on **Test Driven Development** *"Test-Driven Development (TDD) is a methodology in software development that necessitates tests to be written before the production code. This approach can be used in any software development paradigm that involves writing code, including Agile, Scrum, XP, and Lean."* (Agha et al., 2023).



9. Conclusion and Recommendations

The project is set out to answer the following problem statement:

"How can an organizer and participants work together to share the burden of event creation among themselves and streamline the management of an event?"

When looking at the analysis, design, implementation and testing, and the fact that every important requirement was implemented, it can be concluded that the project fulfills the problem statement. The requirements were ranked on importance based on how much they would affect the main problem statement and sub-questions. The main features such as party creation, date voting, item contribution and the chat system combined in one place, supports the collaborative approach that the Party Planner application seeks to accomplish in order to lift the weight on the organizers shoulders.

From an ethical perspective the system should have a layer of security since it handles personal information in the form of login information. Therefore, the system implements hash passing for the passwords in the database. Even though password hashing was added, many security features lack which should be considered for future projects.

Based on this information, it is safe to conclude that the core functionality of the application was a success, but here are some of the recommendations that would further fulfil the project:

- First the remaining 2 functional requirements could be implemented in order to fully complete the system and not have the excess data that the userId adds.
- Secondly the use cases could have been tested more thoroughly with the ZOMBIES testing instead of Blackbox to ensure a complete evaluation of the system.
- Thirdly, additional features could be added to further improve the systems overall security.



10. Appendix

Appendix A - Project Description and Inception

- Project Description.pdf
- Inception.pdf

Appendix B - Contract and Collaboration

- Group Contract.pdf
- Cultural Map.pdf
- Personal Cultural Profiles

Appendix C - Sprint Management

- Sprint Documentation.pdf
- Burn Down Charts

Appendix D - Use Case Descriptions

- Use Cases Description.pdf

Appendix E - UML Diagram Exports (pdfs and pngs)

- 5.1 Use Case Diagram
- 5.2 Domain Model
- 5.3 Class Diagram
- 5.4 Activity Diagrams
- 5.5 Sequence Diagrams
- 5.6 GR Diagram
- 5.7 MVVM Diagram
- 5.8 EER Diagram

Appendix F - Astah Source Files

- 6.1 Use Case Diagram
- 6.2 Domain Model
- 6.3 Class Diagram
- 6.4 Activity Diagrams
- 6.5 Sequence Diagrams
- 6.6 GR Diagram
- 6.7 MVVM Diagram
- 6.8 EER Diagram



Appendix G - Database Source File

- party_planner.sql

Appendix H - User Guide

- guide 3.mp4

Appendix I - Program Source Code

- PartyPlannerSep2.zip

11. References

Agha, D., Sohail, R., Meghji, A. F., Qaboolio, R., & Bhatti, S. (2023). Test driven development and its impact on program design and software quality: A Systematic literature review.

VAWKUM Transactions on Computer Sciences, 11(1), 268–280.

<https://doi.org/10.21015/vtcs.v11i1.1494>

Janssen, T. (2024, March 15). *SOLID Design Principles Explained: Dependency Inversion Principle with Code Examples*. Stackify. <https://stackify.com/dependency-inversion-principle/>

Tervooren, T. (2019, January 7). How to manage your overwhelming social Calendar. Riskology. <https://www.riskology.co/introvert-social-calendar/>

Lesson 8: Object-Oriented Programming. (n.d.).

<https://www.oracle.com/java/technologies/oop.html>

Goetz, B., Peierls, T., Bloch, J., Bowbeer, J., Holmes, D., & Lea, D. (2006). *Java concurrency in practice*. Addison-Wesley.

Gossman, J. (2005, October 8). *Introduction to Model/View/ViewModel pattern for building WPF apps*. Microsoft Developer Blogs.

<https://learn.microsoft.com/en-us/archive/blogs/johngossman/introduction-to-modelviewviewmodel-pattern-for-building-wpf-apps>



Grenning, J. (n.d.). *TDD guided by ZOMBIES*. James Grenning's Blog.

<http://blog.wingman-sw.com/tdd-guided-by-zombies>

Kurose, J. F., & Ross, K. W. (2021). *Computer networking: A top-down approach* (8th ed.). Pearson.

Mockito. (n.d.). *Mockito framework site*. <https://site.mockito.org/>

Oracle. (n.d.). *Core J2EE patterns – Data access object*.

<https://www.oracle.com/java/technologies/dataaccessobject.html>

The PostgreSQL Global Development Group. (n.d.). *PostgreSQL documentation*.

<https://www.postgresql.org/docs/>